

ĐẠI HỌC KINH TẾ THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG CÔNG NGHỆ VÀ THIẾT KẾ
KHOA CÔNG NGHỆ THÔNG TIN KINH DOANH



HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU ORACLE TABLESPACE, TRANSACTION VÀ LOCK (*CONSISTENCY, NONCURRENCY*)

Môn học: Quản trị cơ sở dữ liệu

Giảng viên hướng dẫn: Ths. Phan Hiền

Mã lớp sinh viên: CY0001 - An toàn thông tin - K50

Danh sách sinh viên nhóm:

Nguyễn Minh Quang (<i>Trưởng nhóm</i>)	- 31241021666
Nguyễn Ngọc Quỳnh Chi	- 31241025188
Nguyễn Ngọc Tuyết Ngân	- 31241023809
Nguyễn Đức Thuận	- 31241024347
Nguyễn Thị Phương Trâm	- 31241023913
Huỳnh Hồng Ánh Tuyết	- 31241021807

Thành phố Hồ Chí Minh, ngày 12 tháng 04 năm 2026

LỜI NÓI ĐẦU

Dữ liệu ngày càng giữ vai trò trung tâm trong hoạt động của các hệ thống thông tin, không chỉ về mặt lưu trữ mà còn ở khả năng xử lý và khai thác hiệu quả. Khi số lượng người dùng và tần suất truy cập tăng lên, các hệ quản trị cơ sở dữ liệu phải đối mặt với yêu cầu đồng thời đảm bảo tính chính xác của dữ liệu và duy trì hiệu năng xử lý ổn định. Điều này đặt ra bài toán không đơn thuần nằm ở việc tổ chức dữ liệu, mà còn ở cách hệ thống kiểm soát giao dịch và điều phối truy cập trong môi trường đa người dùng.

Oracle là một trong những hệ quản trị cơ sở dữ liệu tiêu biểu cho cách tiếp cận này, với kiến trúc được xây dựng xoay quanh các cơ chế cốt lõi như Tablespace, Transaction và Lock. Mỗi thành phần đảm nhận một vai trò riêng, nhưng chỉ thực sự phát huy hiệu quả khi được vận hành trong mối liên kết chặt chẽ: Tablespace tổ chức không gian lưu trữ, Transaction điều phối các thay đổi dữ liệu theo nguyên lý ACID, trong khi Lock kiểm soát truy cập nhằm hạn chế xung đột và đảm bảo tính nhất quán.

Dựa trên hướng tiếp cận đó, đề tài “Hệ quản trị cơ sở dữ liệu Oracle với các cơ chế Tablespace, Transaction và Lock (Consistency, Concurrency)” được thực hiện nhằm làm rõ cách các thành phần này tương tác trong thực tế. Nội dung nghiên cứu không dừng lại ở việc trình bày khái niệm, mà tập trung phân tích cơ chế vận hành và kiểm chứng thông qua các kịch bản thực nghiệm trên Oracle 19c, qua đó làm nổi bật vai trò của từng thành phần trong việc duy trì sự ổn định và độ tin cậy của hệ thống.

Thông qua quá trình thực hiện, nhóm hướng đến việc củng cố nền tảng kiến thức về quản trị cơ sở dữ liệu, đồng thời rút ra những hàm ý thực tiễn phục vụ cho việc thiết kế và vận hành các hệ thống có yêu cầu cao về xử lý đồng thời. Dù đã có nhiều nỗ lực, bài tiểu luận vẫn khó tránh khỏi những hạn chế nhất định, nhóm rất mong nhận được sự góp ý từ giảng viên để hoàn thiện hơn trong các nghiên cứu tiếp theo.

MỤC LỤC

LỜI NÓI ĐẦU	i
MỤC LỤC	ii
DANH MỤC HÌNH ẢNH	iv
DANH MỤC BẢNG BIỂU	v
A. PHẦN MỞ ĐẦU	1
1. Lý do chọn đề tài:.....	1
2. Mục tiêu hướng đến:	1
B. PHẦN NỘI DUNG.....	3
CHƯƠNG 1. TỔNG QUAN VỀ KIẾN TRÚC LOGIC VÀ CƠ CHẾ ĐỒNG THỜI.....	3
1.1. Từ lưu trữ vật lý đến quản lý logic:	3
1.2. Nguyên lý ACID trong giao dịch:	4
1.3. Multiversion Concurrency Control (MVCC):.....	6
CHƯƠNG 2. QUẢN TRỊ KHÔNG GIAN LƯU TRỮ LOGIC (TABLESPACE) ..	8
2.1. Khái niệm và vai trò của Tablespace:	8
2.2. Phân loại Tablespace trong Oracle 19c:	11
2.3. Cơ chế quản lý dung lượng và Quota:	16
CHƯƠNG 3. QUẢN LÝ GIAO DỊCH (TRANSACTION) VÀ CƠ CHẾ ĐẢM BẢO TÍNH TOÀN VỆ DỮ LIỆU TRONG ORACLE	19
3.1. Giao dịch như một đơn vị logic trong đảm bảo toàn vẹn dữ liệu:	19
3.2. Cơ chế Undo và Redo - Hai trụ cột của tính nhất quán và khả năng phục hồi: ..	20
3.3. Điều khiển giao dịch - Cơ chế can thiệp vào trạng thái nhất quán:	22
3.4. Tính cô lập và nhất quán trong môi trường đa giao dịch:	22
3.5. Đánh giá và hàm ý thực tiễn:	23
CHƯƠNG 4. CƠ CHẾ KHÓA (LOCK) VÀ ĐIỀU KHIỂN ĐỒNG THỜI	24
4.1. Khái niệm và mục đích của Lock:	24
4.2. Phân loại cơ chế Lock:	26
4.3. Hiện tượng chờ khóa và khóa chết:	27
CHƯƠNG 5. TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ	30
5.1. Thực nghiệm quản trị Tablespace:	30
5.2. Thực nghiệm kiểm soát TRANSACTION và tính toàn vẹn:	33
5.3. Thực nghiệm xử lý đồng thời (Concurrency) và hiện tượng chờ khóa (Blocking):	37

5.4. Thực nghiệm hiện tượng Deadlock (Khóa chết):.....	41
5.5. Nhận xét và đánh giá tổng quan:.....	43
C. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN.....	45
TÀI LIỆU THAM KHẢO.....	47
PHỤ LỤC	49

DANH MỤC HÌNH ẢNH

Hình 3.1: Quy trình xử lý giao dịch trong Oracle với cơ chế Undo và Redo	20
Hình 4.1: Khái niệm và mục đích của Lock.....	25
Hình 4.2: Phạm vi tác động của khóa dòng và khóa bảng	27
Hình 4.3: Sơ đồ minh họa trạng thái chờ đợi vòng tròn dẫn đến khóa chết.....	29
Hình 5.1: Kết quả thực thi lệnh Create Tablespace.	31
Hình 5.2: Kết quả thực thi lệnh Create Table.	32
Hình 5.3: Kết quả thực thi lệnh Alter Database Datafile.....	32
Hình 5.4: Kết quả thực thi lệnh cấp quyền Quota Unlimited.....	33
Hình 5.5: Kết quả thực thi lệnh Insert.	34
Hình 5.6: Kết quả thực thi lệnh COMMIT.	35
Hình 5.7: Quá trình thực thi lệnh ROLLBACK TO SAVEPOINT.	37
Hình 5.8: Kết quả khôi phục số dư tài khoản.	37
Hình 5.9: Lệnh Update tại Phiên A thiết lập khóa độc quyền (Row-level Lock).....	38
Hình 5.10: Truy vấn bị chờ (Blocking) tại Phiên B	39
Hình 5.11: Lệnh tại Phiên B thực thi thành công ngay sau khi khóa được giải phóng.	40
Hình 5.12: Kết quả sau khi giải phóng khóa và thực thi thành công	40
Hình 5.13: Truy vấn bị treo do cập nhật chéo.	42
Hình 5.14: Oracle phát hiện deadlock và hủy giao dịch.	42

DANH MỤC BẢNG BIỂU

Bảng 2.1: So sánh tổng hợp Tablespace trong Oracle	16
Bảng 5.1: Đối chiếu kết quả thực nghiệm xử lý xung đột.....	44

A. PHẦN MỞ ĐẦU

1. Lý do chọn đề tài:

Sự gia tăng theo cấp số nhân của dữ liệu và tần suất truy cập trong các hệ thống thông tin hiện đại đã đặt ra yêu cầu ngày càng khắt khe đối với việc tổ chức lưu trữ và kiểm soát truy cập dữ liệu. Khi hàng nghìn, thậm chí hàng triệu giao dịch có thể diễn ra đồng thời trên cùng một hệ thống, nguy cơ phát sinh xung đột, sai lệch dữ liệu hoặc suy giảm hiệu năng trở thành một thách thức mang tính bản chất. Điều này không chỉ ảnh hưởng đến tính toàn vẹn của dữ liệu mà còn tác động trực tiếp đến độ tin cậy và khả năng vận hành của toàn bộ hệ thống.

Trong bối cảnh đó, việc nghiên cứu các cơ chế quản trị không gian lưu trữ và kiểm soát đồng thời không còn dừng lại ở mức lý thuyết, mà trở thành một yêu cầu thiết yếu trong thiết kế và vận hành cơ sở dữ liệu. Đặc biệt, hệ quản trị cơ sở dữ liệu Oracle 19c với các thành phần như Tablespace, Transaction và Lock đã cung cấp một kiến trúc hoàn chỉnh nhằm giải quyết đồng thời nhiều bài toán: từ tổ chức dữ liệu ở mức logic - vật lý, đảm bảo tính toàn vẹn theo nguyên lý ACID, đến điều phối truy cập trong môi trường đa người dùng. Chính vì vậy, việc lựa chọn nghiên cứu đề tài này không chỉ mang ý nghĩa học thuật mà còn có giá trị thực tiễn cao, giúp làm rõ cách các cơ chế nội tại của Oracle phối hợp để duy trì sự ổn định và nhất quán của hệ thống.

2. Mục tiêu hướng đến:

Đề tài hướng đến việc xây dựng một cái nhìn hệ thống và toàn diện về các cơ chế cốt lõi trong quản trị cơ sở dữ liệu Oracle, thông qua việc kết hợp giữa phân tích lý thuyết và triển khai thực nghiệm. Trọng tâm nghiên cứu tập trung vào ba thành phần chính: Tablespace - đại diện cho cơ chế quản trị không gian lưu trữ logic; Transaction - nền tảng đảm bảo tính toàn vẹn dữ liệu theo mô hình ACID; và Lock - công cụ điều phối truy cập nhằm kiểm soát xung đột trong môi trường xử lý đồng thời.

Không dừng lại ở việc mô tả khái niệm, đề tài tiến hành xây dựng các kịch bản thực nghiệm trực tiếp trên Oracle 19c nhằm kiểm chứng cách các cơ chế này vận hành trong thực tế. Thông qua các tình huống như cấp phát và mở rộng Tablespace, thực thi

và kiểm soát giao dịch, cũng như mô phỏng hiện tượng blocking và deadlock, nghiên cứu làm rõ mối liên hệ giữa lý thuyết và hành vi hệ thống. Từ đó, đề tài không chỉ giúp củng cố nền tảng kiến thức mà còn cung cấp những hàm ý thực tiễn trong việc thiết kế, tối ưu và vận hành các hệ thống cơ sở dữ liệu có yêu cầu cao về hiệu năng và độ tin cậy.

B. PHẦN NỘI DUNG

CHƯƠNG 1. TỔNG QUAN VỀ KIẾN TRÚC LOGIC VÀ CƠ CHẾ ĐỒNG THỜI

Trong các hệ quản trị cơ sở dữ liệu hiện đại, việc quản lý dữ liệu không chỉ dừng lại ở lưu trữ vật lý mà còn đòi hỏi một hệ thống tổ chức và kiểm soát ở mức logic. Sự kết hợp giữa hai lớp này giúp hệ thống vận hành hiệu quả, đồng thời đảm bảo dữ liệu luôn nhất quán và an toàn trong môi trường có nhiều truy cập đồng thời.

1.1. Từ lưu trữ vật lý đến quản lý logic:

Trong các hệ quản trị cơ sở dữ liệu hiện đại, dữ liệu không chỉ đơn thuần được lưu trữ dưới dạng các tệp trên thiết bị vật lý, mà còn được tổ chức thông qua một lớp quản lý logic nhằm hỗ trợ việc truy xuất, kiểm soát và tối ưu hóa hiệu năng hệ thống. Việc phân tách giữa lưu trữ vật lý và quản lý logic cho phép hệ quản trị cơ sở dữ liệu vận hành linh hoạt hơn, đồng thời giảm sự phụ thuộc của người dùng vào các chi tiết kỹ thuật ở mức thấp.

Ở mức vật lý, dữ liệu được lưu trữ trong các datafile, là các tệp tồn tại trên hệ thống lưu trữ. Bên trong mỗi datafile, dữ liệu được chia thành các block (khối dữ liệu), đây là đơn vị nhỏ nhất mà hệ quản trị cơ sở dữ liệu có thể thực hiện thao tác đọc hoặc ghi. Cách tổ chức này giúp tối ưu hóa quá trình truy xuất dữ liệu, bởi hệ thống thực hiện các thao tác I/O theo từng khối thay vì từng bản ghi riêng lẻ.

Tuy nhiên, để thuận tiện cho việc quản trị và sử dụng, hệ quản trị cơ sở dữ liệu cung cấp một lớp trừu tượng ở mức logic. Ở mức này, dữ liệu được tổ chức thành các tablespace, đóng vai trò như không gian lưu trữ logic, cho phép phân chia dữ liệu theo từng nhóm hoặc chức năng cụ thể. Bên trong mỗi tablespace là các segment, đại diện cho các đối tượng logic như bảng (table), chỉ mục (index) hoặc các cấu trúc dữ liệu khác. Mỗi segment tiếp tục được chia thành nhiều extent, và mỗi extent bao gồm một tập hợp các block liên tiếp.

Như vậy, cấu trúc lưu trữ của hệ quản trị cơ sở dữ liệu được tổ chức theo một hệ phân cấp rõ ràng: một tablespace bao gồm nhiều segment; mỗi segment bao gồm nhiều extent; mỗi extent được hình thành từ nhiều block; và các block này được lưu trữ vật lý trong các datafile. Cơ chế ánh xạ này cho phép hệ thống tách biệt giữa cách dữ liệu được lưu trữ thực tế và cách dữ liệu được biểu diễn, từ đó nâng cao khả năng mở rộng và tối ưu hóa hiệu năng truy xuất.

Trong thực tế, khi người dùng thao tác với một bảng dữ liệu, chẳng hạn như bảng “SinhVien”, toàn bộ các thao tác như truy vấn, thêm, sửa hoặc xóa dữ liệu đều được thực hiện thông qua lớp logic mà không cần quan tâm đến vị trí lưu trữ vật lý của dữ liệu. Tuy nhiên, chính sự tách biệt này lại đặt ra một yêu cầu quan trọng: hệ quản trị cơ sở dữ liệu phải đảm bảo rằng mọi thao tác trên dữ liệu, dù diễn ra ở mức logic, vẫn được thực thi một cách chính xác, nhất quán và an toàn ở mức vật lý.

Đặc biệt, trong môi trường có nhiều người dùng truy cập đồng thời, các thao tác đọc và ghi dữ liệu có thể diễn ra cùng lúc trên cùng một tập dữ liệu. Nếu không có cơ chế kiểm soát phù hợp, điều này có thể dẫn đến sai lệch dữ liệu, mất tính nhất quán hoặc thậm chí làm hỏng hệ thống. Do đó, bên cạnh việc tổ chức dữ liệu hiệu quả, hệ quản trị cơ sở dữ liệu cần có các nguyên lý và cơ chế nhằm đảm bảo tính toàn vẹn và an toàn cho các giao dịch dữ liệu.

Chính trong bối cảnh đó, các nguyên lý như ACID được xây dựng nhằm đảm bảo rằng mọi giao dịch trong hệ thống đều được thực hiện một cách đáng tin cậy, bất chấp các yếu tố như lỗi hệ thống hay truy cập đồng thời từ nhiều người dùng.

1.2. Nguyên lý ACID trong giao dịch:

Như đã đề cập ở phần trước, mặc dù hệ quản trị cơ sở dữ liệu cung cấp một lớp trừu tượng logic giúp người dùng thao tác với dữ liệu một cách thuận tiện, nhưng các thao tác này thực chất vẫn tác động trực tiếp đến dữ liệu được lưu trữ ở mức vật lý. Trong môi trường có nhiều người dùng truy cập đồng thời, việc thực hiện các thao tác đọc và ghi dữ liệu có thể dẫn đến những vấn đề như mất nhất quán, xung đột dữ liệu hoặc lỗi hệ thống nếu không được kiểm soát chặt chẽ. Do đó, việc đảm bảo tính đúng

đảm bảo và an toàn cho các giao dịch dữ liệu trở thành một yêu cầu cốt lõi trong thiết kế hệ quản trị cơ sở dữ liệu.

Để giải quyết vấn đề này, các hệ quản trị cơ sở dữ liệu hiện đại áp dụng nguyên lý ACID, bao gồm bốn đặc tính cơ bản: Atomicity (tính nguyên tử), Consistency (tính nhất quán), Isolation (tính cô lập) và Durability (tính bền vững). Bốn đặc tính này phối hợp với nhau nhằm đảm bảo rằng mọi giao dịch được thực hiện một cách đáng tin cậy, ngay cả trong các tình huống có lỗi xảy ra hoặc có nhiều giao dịch diễn ra đồng thời.

Trước hết, Atomicity (tính nguyên tử) đảm bảo rằng một giao dịch được thực hiện theo nguyên tắc “tất cả hoặc không gì cả”. Điều này có nghĩa là nếu bất kỳ thao tác nào trong giao dịch thất bại, toàn bộ giao dịch sẽ bị hủy bỏ và hệ thống sẽ quay trở về trạng thái ban đầu trước khi giao dịch được thực hiện. Ví dụ, trong một hệ thống ngân hàng, khi thực hiện chuyển tiền từ tài khoản A sang tài khoản B, hệ thống phải đảm bảo rằng việc trừ tiền ở tài khoản A và cộng tiền vào tài khoản B diễn ra đồng thời; nếu một trong hai thao tác thất bại, toàn bộ giao dịch sẽ không được ghi nhận.

Tiếp theo, Consistency (tính nhất quán) đảm bảo rằng cơ sở dữ liệu luôn chuyển từ một trạng thái hợp lệ sang một trạng thái hợp lệ khác sau mỗi giao dịch. Điều này đồng nghĩa với việc mọi ràng buộc dữ liệu, chẳng hạn như khóa chính, khóa ngoại hoặc các quy tắc nghiệp vụ, luôn được duy trì. Nếu một giao dịch vi phạm các ràng buộc này, hệ thống sẽ từ chối thực hiện giao dịch đó nhằm bảo vệ tính toàn vẹn của dữ liệu.

Isolation (tính cô lập) đề cập đến khả năng đảm bảo rằng các giao dịch thực thi đồng thời không ảnh hưởng lẫn nhau theo cách gây sai lệch dữ liệu. Mỗi giao dịch được thực hiện như thể nó là giao dịch duy nhất trong hệ thống, mặc dù trên thực tế có thể có nhiều giao dịch đang diễn ra cùng lúc. Mức độ cô lập có thể khác nhau tùy theo cấu hình của hệ quản trị cơ sở dữ liệu, nhưng mục tiêu chung là ngăn chặn các hiện tượng bất thường như đọc dữ liệu chưa được COMMIT (dirty read), đọc không lặp lại (non-repeatable read) hoặc xuất hiện dữ liệu ảo (phantom read).

Cuối cùng, Durability (tính bền vững) đảm bảo rằng một khi giao dịch đã được xác nhận (COMMIT), các thay đổi của nó sẽ được lưu trữ vĩnh viễn trong hệ thống, ngay cả khi xảy ra sự cố như mất điện hoặc lỗi phần cứng. Điều này thường được thực hiện

thông qua các cơ chế ghi log và phục hồi dữ liệu, cho phép hệ thống khôi phục lại trạng thái đã được COMMIT trước đó.

Nhìn chung, nguyên lý ACID đóng vai trò nền tảng trong việc đảm bảo tính toàn vẹn và độ tin cậy của dữ liệu trong các hệ quản trị cơ sở dữ liệu. Tuy nhiên, để hiện thực hóa đặc tính Isolation trong môi trường có nhiều giao dịch đồng thời mà vẫn duy trì hiệu năng cao, các hệ quản trị cơ sở dữ liệu cần áp dụng những cơ chế kiểm soát đồng thời hiệu quả hơn.

Một trong những cơ chế nổi bật được sử dụng trong các hệ quản trị cơ sở dữ liệu như Oracle là Multiversion Concurrency Control (MVCC), cho phép nhiều giao dịch có thể đọc và ghi dữ liệu đồng thời mà không gây ra xung đột trực tiếp. Cơ chế này sẽ được trình bày chi tiết trong phần tiếp theo.

1.3. Multiversion Concurrency Control (MVCC):

Như đã trình bày ở phần trước, một trong những thách thức lớn trong hệ quản trị cơ sở dữ liệu là đảm bảo tính cô lập (Isolation) của các giao dịch trong khi vẫn duy trì hiệu năng cao khi có nhiều người dùng truy cập đồng thời. Nếu áp dụng cơ chế khóa truyền thống (locking) một cách nghiêm ngặt, các giao dịch đọc có thể bị chặn bởi các giao dịch ghi và ngược lại, dẫn đến tình trạng giảm hiệu suất và gia tăng độ trễ của hệ thống.

Để khắc phục hạn chế này, các hệ quản trị cơ sở dữ liệu hiện đại, đặc biệt là Oracle, áp dụng cơ chế Multiversion Concurrency Control (MVCC). Cơ chế này cho phép nhiều phiên bản của cùng một dữ liệu tồn tại đồng thời, từ đó giúp các giao dịch đọc và ghi có thể diễn ra song song mà không gây xung đột trực tiếp.

Nguyên lý cốt lõi của MVCC là thay vì ghi đè trực tiếp lên dữ liệu hiện tại khi có thao tác cập nhật, hệ thống sẽ tạo ra một phiên bản mới của dữ liệu. Phiên bản cũ vẫn được giữ lại trong một khoảng thời gian nhất định nhằm phục vụ các giao dịch đang cần truy cập dữ liệu ở trạng thái trước đó. Nhờ vậy, mỗi giao dịch có thể “nhìn thấy” một ảnh chụp (snapshot) nhất quán của dữ liệu tại thời điểm nó bắt đầu, mà không bị ảnh hưởng bởi các thay đổi chưa được COMMIT từ các giao dịch khác.

Trong Oracle, cơ chế này được hỗ trợ thông qua việc sử dụng vùng dữ liệu UNDO. Khi một bản ghi bị thay đổi, giá trị cũ của bản ghi sẽ được lưu vào vùng UNDO. Nếu một giao dịch cần đọc dữ liệu tại thời điểm trước khi thay đổi xảy ra, hệ thống sẽ sử dụng thông tin trong UNDO để tái tạo lại phiên bản dữ liệu phù hợp với thời điểm đó. Điều này cho phép các giao dịch đọc không bị chặn bởi các giao dịch ghi đang diễn ra.

Một đặc điểm quan trọng của MVCC là nguyên tắc: “reader không chặn writer, và writer không chặn reader”. Cụ thể, các truy vấn đọc dữ liệu sẽ không cần phải chờ các giao dịch ghi hoàn tất, trong khi các thao tác ghi cũng không bị trì hoãn bởi các truy vấn đọc. Nhờ đó, hệ thống có thể xử lý đồng thời nhiều giao dịch với hiệu năng cao mà vẫn đảm bảo tính nhất quán dữ liệu theo yêu cầu của ACID.

Ví dụ, giả sử người dùng A thực hiện truy vấn đọc dữ liệu tại một thời điểm nhất định, trong khi người dùng B đồng thời cập nhật cùng dữ liệu đó. Với cơ chế MVCC, người dùng A vẫn sẽ nhận được phiên bản dữ liệu tại thời điểm truy vấn bắt đầu, trong khi người dùng B tiếp tục thực hiện cập nhật mà không gây ảnh hưởng đến kết quả của A. Sau khi giao dịch của B được COMMIT, các giao dịch mới sẽ nhìn thấy dữ liệu đã được cập nhật, trong khi các giao dịch cũ vẫn duy trì tính nhất quán của mình.

Nhờ cơ chế kiểm soát đa phiên bản, MVCC không chỉ giúp cải thiện đáng kể hiệu năng của hệ thống trong môi trường đa người dùng, mà còn đóng vai trò quan trọng trong việc hiện thực hóa đặc tính Isolation của nguyên lý ACID. Điều này cho thấy sự kết hợp chặt chẽ giữa các nguyên lý giao dịch và cơ chế triển khai cụ thể trong hệ quản trị cơ sở dữ liệu, góp phần đảm bảo hệ thống vừa hoạt động hiệu quả, vừa duy trì được tính toàn vẹn và độ tin cậy của dữ liệu.

CHƯƠNG 2. QUẢN TRỊ KHÔNG GIAN LƯU TRỮ LOGIC (TABLESPACE)

2.1. Khái niệm và vai trò của Tablespace:

2.1.1. Khái niệm Tablespace:

Trong hệ quản trị cơ sở dữ liệu Oracle, Tablespace được xem là đơn vị lưu trữ logic cao nhất trong cấu trúc quản lý dữ liệu, đóng vai trò trung tâm trong việc tổ chức, phân bổ và kiểm soát tài nguyên lưu trữ. Về bản chất, Tablespace là một lớp trừu tượng (abstraction layer) nằm giữa tầng logic (logical storage layer) và tầng vật lý (physical storage layer), cho phép hệ thống ánh xạ các đối tượng dữ liệu như bảng (table), chỉ mục (index) hay các cấu trúc dữ liệu khác xuống các file vật lý (datafile) một cách linh hoạt và hiệu quả.

Khác với cách lưu trữ trực tiếp vào file như trong hệ thống tệp truyền thống, Oracle sử dụng Tablespace như một cơ chế trung gian nhằm tách biệt hoàn toàn giữa cách dữ liệu được tổ chức logic và cách dữ liệu được lưu trữ vật lý. Mỗi Tablespace có thể bao gồm một hoặc nhiều datafile, và các datafile này có thể được phân bổ trên nhiều thiết bị lưu trữ khác nhau. Chính sự tách biệt này cho phép hệ thống dễ dàng mở rộng, tái cấu trúc hoặc tối ưu hóa lưu trữ mà không ảnh hưởng đến cấu trúc logic của dữ liệu.

Ở mức độ nội tại, Tablespace không chỉ đơn thuần là một “container” chứa dữ liệu, mà còn là môi trường quản lý không gian thông qua các đơn vị như segment, extent và block. Khi một đối tượng dữ liệu được tạo ra, hệ thống sẽ cấp phát không gian từ Tablespace dưới dạng các extent, và quá trình này được quản lý tự động thông qua các cơ chế như Locally Managed Tablespace (LMT). Nhờ đó, việc phân bổ và thu hồi không gian trở nên hiệu quả hơn, giảm thiểu phân mảnh và cải thiện hiệu năng truy xuất dữ liệu.

2.1.2. Vai trò của Tablespace:

Vai trò của Tablespace không chỉ dừng lại ở việc lưu trữ dữ liệu mà còn thể hiện rõ trong nhiều khía cạnh quan trọng của quản trị và vận hành hệ thống:

2.1.2.1. Tổ chức dữ liệu hiệu quả:

- Tablespace cho phép nhóm các đối tượng dữ liệu có liên quan vào cùng một không gian lưu trữ, từ đó tạo ra một cấu trúc tổ chức dữ liệu mang tính hệ thống và dễ quản lý.

- Ví dụ, các bảng liên quan đến một module nghiệp vụ cụ thể có thể được đặt trong cùng một Tablespace, giúp việc theo dõi, bảo trì và tối ưu trở nên thuận tiện hơn.

- Quan trọng hơn, việc tổ chức dữ liệu theo Tablespace còn hỗ trợ cho các chiến lược phân vùng (partitioning), nơi dữ liệu lớn được chia thành nhiều phần nhỏ và lưu trữ trên các Tablespace khác nhau. Điều này giúp cải thiện đáng kể hiệu năng truy vấn và khả năng mở rộng của hệ thống.

2.1.2.2. Tối ưu hóa hiệu năng hệ thống:

- Tablespace đóng vai trò quan trọng trong việc tối ưu hóa hiệu năng, đặc biệt liên quan đến hoạt động I/O. Bằng cách phân bổ các Tablespace trên các thiết bị lưu trữ khác nhau (ví dụ: nhiều ổ đĩa), hệ thống có thể thực hiện truy xuất dữ liệu song song, giảm thiểu tình trạng nghẽn I/O.

- Ngoài ra, việc tách biệt các loại dữ liệu (ví dụ: dữ liệu chính, dữ liệu tạm, undo) vào các Tablespace riêng biệt còn giúp hệ thống tối ưu hóa chiến lược truy cập và xử lý. Điều này đặc biệt quan trọng trong các hệ thống có tải cao hoặc xử lý dữ liệu lớn.

2.1.2.3. Hỗ trợ quản trị và bảo trì hệ thống:

- Tablespace cung cấp cho quản trị viên khả năng kiểm soát linh hoạt trạng thái của dữ liệu. Một Tablespace có thể được đưa vào trạng thái read-only hoặc offline mà không ảnh hưởng đến toàn bộ cơ sở dữ liệu.

- Điều này rất hữu ích trong các hoạt động bảo trì, sao lưu hoặc di chuyển dữ liệu. Hơn nữa, Tablespace còn là đơn vị cơ bản trong các chiến lược backup và recovery.

- Các công cụ như RMAN thường hoạt động dựa trên Tablespace, cho phép thực hiện sao lưu hoặc phục hồi theo từng phần thay vì toàn bộ hệ thống, từ đó tiết kiệm thời gian và tài nguyên.

2.1.2.4 Phân quyền sử dụng và kiểm soát tài nguyên:

- Thông qua cơ chế quota, Tablespace cho phép quản trị viên kiểm soát dung lượng mà mỗi người dùng có thể sử dụng.

- Điều này giúp ngăn chặn tình trạng một user chiếm dụng quá nhiều tài nguyên, đồng thời đảm bảo sự công bằng trong môi trường đa người dùng.

- Ngoài ra, việc phân bổ user vào các Tablespace khác nhau còn giúp thực hiện các chính sách quản trị dữ liệu linh hoạt, phù hợp với vai trò và nhu cầu của từng nhóm người dùng trong hệ thống.

2.1.2.5. Nền tảng cho khả năng mở rộng và linh hoạt:

- Một vai trò quan trọng khác của Tablespace là hỗ trợ khả năng mở rộng của hệ thống. Khi nhu cầu lưu trữ tăng lên, quản trị viên có thể dễ dàng thêm datafile vào Tablespace mà không cần thay đổi cấu trúc logic của dữ liệu.

- Điều này cho phép hệ thống thích ứng linh hoạt với sự gia tăng dữ liệu theo thời gian, đặc biệt trong các hệ thống lớn như ngân hàng, thương mại điện tử hoặc hệ thống phân tích dữ liệu.

Như vậy, Tablespace không chỉ đơn thuần là một đơn vị lưu trữ logic mà còn là một thành phần cốt lõi trong kiến trúc của hệ quản trị cơ sở dữ liệu Oracle. Nó đóng vai trò kết nối giữa tầng logic và vật lý, đồng thời cung cấp các cơ chế quan trọng cho việc tổ chức dữ liệu, tối ưu hiệu năng, quản trị hệ thống và kiểm soát tài nguyên.

Việc hiểu rõ và khai thác hiệu quả Tablespace là một trong những yếu tố then chốt giúp nâng cao hiệu quả vận hành và độ tin cậy của hệ thống cơ sở dữ liệu trong thực tế.

2.2. Phân loại Tablespace trong Oracle 19c:

Trong Oracle 19c, Tablespace không chỉ là đơn vị phân chia không gian lưu trữ mà còn là thành phần quan trọng trong kiến trúc quản trị dữ liệu, đóng vai trò kết nối giữa tầng logic và tầng vật lý của hệ thống. Việc phân loại Tablespace thành Permanent, Temporary và Undo không chỉ nhằm mục đích tổ chức dữ liệu mà còn phản ánh cách hệ quản trị cơ sở dữ liệu tối ưu hóa hiệu năng, kiểm soát giao dịch và đảm bảo tính toàn vẹn dữ liệu trong môi trường đa người dùng. Mỗi loại Tablespace tương ứng với một lớp chức năng riêng biệt, góp phần tạo nên một hệ thống lưu trữ có tính linh hoạt, ổn định và khả năng mở rộng cao.

2.2.1. Permanent Tablespace:

Permanent Tablespace là loại Tablespace quan trọng nhất trong hệ thống, chịu trách nhiệm lưu trữ dữ liệu nghiệp vụ chính dưới dạng bền vững. Đây là nơi chứa các đối tượng dữ liệu như bảng, chỉ mục, phân vùng dữ liệu và các cấu trúc liên quan, đóng vai trò trung tâm trong toàn bộ kiến trúc cơ sở dữ liệu.

2.2.1.1. Đặc điểm:

Một đặc điểm nền tảng của Permanent Tablespace là cơ chế lưu trữ dữ liệu bền vững dựa trên sự phối hợp giữa bộ nhớ và thiết bị lưu trữ vật lý. Khi một giao dịch được xác nhận, dữ liệu không được ghi trực tiếp xuống đĩa ngay lập tức mà thông qua buffer cache, sau đó mới được ghi xuống datafile bởi tiến trình DBWR. Đồng thời, redo log ghi nhận mọi thay đổi trước khi dữ liệu được ghi vật lý, đảm bảo rằng hệ thống có thể phục hồi ngay cả khi xảy ra lỗi đột ngột. Cơ chế này thể hiện rõ chiến lược “write-ahead logging”, một nguyên lý quan trọng trong thiết kế hệ quản trị cơ sở dữ liệu hiện đại.

Bên cạnh đó, Permanent Tablespace sử dụng cơ chế quản lý không gian thông qua các đơn vị như segment, extent và block. Việc phân chia dữ liệu thành các đơn vị nhỏ giúp hệ thống dễ dàng quản lý, cấp phát và thu hồi không gian lưu trữ, đồng thời giảm thiểu tình trạng phân mảnh. Đặc biệt, trong Oracle 19c, cơ chế Locally

Managed Tablespace (LMT) sử dụng bitmap để quản lý extent, giúp tăng hiệu năng và giảm sự phụ thuộc vào data dictionary.

Một đặc điểm quan trọng khác là khả năng hỗ trợ phân vùng dữ liệu (partitioning). Dữ liệu trong Permanent Tablespace có thể được chia thành nhiều phân vùng dựa trên các tiêu chí khác nhau, giúp tối ưu hóa truy vấn và cải thiện hiệu năng khi xử lý dữ liệu lớn. Điều này đặc biệt hữu ích trong các hệ thống có khối lượng dữ liệu lớn như hệ thống tài chính hoặc thương mại điện tử.

Ngoài ra, Permanent Tablespace còn cho phép thiết lập các thuộc tính như read-only hoặc offline, giúp quản trị viên có thể kiểm soát trạng thái hoạt động của dữ liệu. Điều này hỗ trợ hiệu quả cho các hoạt động bảo trì, sao lưu và tối ưu hóa hệ thống mà không ảnh hưởng đến toàn bộ cơ sở dữ liệu.

2.2.1.2. Chức năng:

Permanent Tablespace thực hiện chức năng lưu trữ dữ liệu nghiệp vụ cốt lõi, đóng vai trò là nền tảng cho mọi hoạt động của hệ thống cơ sở dữ liệu. Tất cả các thông tin quan trọng phục vụ cho hoạt động của tổ chức đều được lưu trữ tại đây, do đó độ tin cậy và tính ổn định của Permanent Tablespace có ảnh hưởng trực tiếp đến toàn bộ hệ thống.

Không chỉ dừng lại ở việc lưu trữ, Permanent Tablespace còn là trung tâm của quá trình xử lý giao dịch. Mọi thao tác cập nhật dữ liệu đều được thực hiện tại đây, thông qua sự phối hợp giữa buffer cache, redo log và undo. Cơ chế này đảm bảo rằng hệ thống có thể duy trì đầy đủ các tính chất của ACID, đặc biệt trong môi trường có nhiều giao dịch đồng thời.

Ngoài ra, Permanent Tablespace cũng đóng vai trò quan trọng trong việc tối ưu hiệu năng truy vấn. Thông qua việc phân bổ dữ liệu hợp lý trên các datafile và thiết bị lưu trữ khác nhau, hệ thống có thể giảm thiểu xung đột I/O và cải thiện tốc độ truy xuất dữ liệu. Trong các hệ thống lớn, việc thiết kế tablespace hợp lý có thể ảnh hưởng trực tiếp đến hiệu suất tổng thể.

Hơn nữa, Permanent Tablespace là thành phần không thể thiếu trong các chiến lược sao lưu và phục hồi dữ liệu. Các công cụ như RMAN sử dụng thông tin từ Tablespace để thực hiện backup và recovery, đảm bảo rằng hệ thống có thể phục hồi nhanh chóng trong các tình huống khẩn cấp. Điều này đặc biệt quan trọng đối với các hệ thống yêu cầu tính sẵn sàng cao.

2.2.2. Temporary Tablespace:

Temporary Tablespace đóng vai trò hỗ trợ trong quá trình xử lý truy vấn bằng cách cung cấp không gian lưu trữ tạm thời cho dữ liệu trung gian.

2.2.2.1. Đặc điểm:

Temporary Tablespace có đặc điểm là sử dụng các temporary segment thay vì segment cố định như Permanent Tablespace. Các segment này được tạo ra khi cần thiết và tự động giải phóng sau khi sử dụng, giúp hệ thống tối ưu hóa việc sử dụng tài nguyên.

Một đặc điểm quan trọng khác là việc không sử dụng redo log cho các thao tác trên Temporary Tablespace. Điều này giúp giảm đáng kể lượng dữ liệu cần ghi xuống đĩa, từ đó cải thiện hiệu năng. Tuy nhiên, điều này cũng đồng nghĩa với việc dữ liệu trong Temporary Tablespace không thể phục hồi, nhưng điều này không gây ảnh hưởng vì dữ liệu chỉ mang tính tạm thời.

Ngoài ra, Temporary Tablespace hoạt động chặt chẽ với bộ nhớ PGA trong quá trình xử lý truy vấn. Khi dữ liệu vượt quá khả năng xử lý của bộ nhớ, hệ thống sẽ tự động chuyển sang sử dụng Temporary Tablespace. Cơ chế này được gọi là “spill to disk”, giúp đảm bảo rằng các truy vấn lớn vẫn có thể được thực hiện mà không bị lỗi do thiếu bộ nhớ.

Một điểm đáng chú ý khác là Temporary Tablespace được quản lý theo session, giúp giảm thiểu xung đột giữa các người dùng. Mỗi session có thể sử dụng không gian tạm riêng, từ đó tăng khả năng xử lý đồng thời của hệ thống.

2.2.2.2. Chức năng:

Temporary Tablespace thực hiện chức năng hỗ trợ thực thi truy vấn bằng cách cung cấp không gian lưu trữ trung gian cho dữ liệu phát sinh trong quá trình xử lý. Đây là yếu tố quan trọng giúp hệ thống có thể xử lý các truy vấn phức tạp một cách hiệu quả.

Thông qua việc đóng vai trò như một vùng mở rộng của bộ nhớ, Temporary Tablespace giúp hệ thống duy trì hiệu năng ổn định trong các tình huống tải cao. Khi nhiều truy vấn được thực thi đồng thời, việc sử dụng Temporary Tablespace giúp giảm áp lực lên bộ nhớ chính và tránh tình trạng nghẽn tài nguyên.

Ngoài ra, Temporary Tablespace còn góp phần tối ưu hóa kế hoạch thực thi truy vấn (execution plan). Trình tối ưu hóa của Oracle có thể quyết định sử dụng không gian tạm để thực hiện các phép toán hiệu quả hơn, từ đó cải thiện tốc độ xử lý.

Trong các hệ thống phân tích dữ liệu lớn (data warehouse), Temporary Tablespace đóng vai trò đặc biệt quan trọng, bởi các truy vấn thường yêu cầu xử lý lượng dữ liệu rất lớn. Việc quản lý tốt Temporary Tablespace sẽ giúp hệ thống đạt hiệu năng cao và ổn định hơn.

2.2.3. Undo Tablespace:

Undo Tablespace giữ vai trò đặc biệt quan trọng trong việc đảm bảo tính toàn vẹn và nhất quán của dữ liệu.

2.2.3.1. Đặc điểm:

Undo Tablespace sử dụng undo segment để lưu trữ các bản ghi thay đổi dữ liệu. Mỗi giao dịch sẽ ghi lại các thay đổi vào undo segment tương ứng, cho phép hệ thống theo dõi và quản lý trạng thái của từng giao dịch.

Một đặc điểm quan trọng là cơ chế quản lý thời gian tồn tại của undo thông qua tham số undo retention. Tham số này quy định thời gian mà dữ liệu undo được giữ lại, ảnh hưởng trực tiếp đến khả năng thực hiện các truy vấn FLASHBACK.

Ngoài ra, Undo Tablespace hỗ trợ cơ chế MVCC, cho phép nhiều người dùng truy cập dữ liệu đồng thời mà không gây xung đột. Thay vì khóa dữ liệu, hệ thống cung cấp các phiên bản dữ liệu khác nhau dựa trên undo, từ đó tăng hiệu năng và giảm độ trễ.

Một điểm đáng chú ý khác là Undo Tablespace có thể trở thành bottleneck nếu không được cấu hình hợp lý. Việc thiếu không gian undo có thể dẫn đến lỗi hoặc làm gián đoạn giao dịch, do đó việc quản lý dung lượng undo là rất quan trọng.

2.2.3.2. Chức năng:

Undo Tablespace thực hiện chức năng hỗ trợ quản lý giao dịch bằng cách cung cấp cơ chế hoàn tác và duy trì trạng thái dữ liệu nhất quán. Đây là yếu tố quan trọng giúp hệ thống đảm bảo tính toàn vẹn dữ liệu trong mọi tình huống.

Thông qua việc lưu trữ các phiên bản trước của dữ liệu, Undo Tablespace cho phép hệ thống thực hiện ROLLBACK một cách chính xác, đồng thời hỗ trợ các truy vấn đọc dữ liệu mà không bị ảnh hưởng bởi các thay đổi chưa hoàn tất.

Ngoài ra, Undo Tablespace còn đóng vai trò quan trọng trong việc nâng cao khả năng xử lý đồng thời của hệ thống. Bằng cách sử dụng cơ chế MVCC, hệ thống có thể cho phép nhiều giao dịch hoạt động song song mà không cần khóa dữ liệu, từ đó cải thiện hiệu năng.

Hơn nữa, Undo Tablespace còn hỗ trợ các chức năng nâng cao như Flashback Query, Flashback Table và Flashback Database, cho phép truy xuất và phục hồi dữ liệu tại các thời điểm trong quá khứ. Điều này mang lại khả năng kiểm soát dữ liệu mạnh mẽ và linh hoạt cho quản trị viên.

2.2.4. So sánh tổng hợp:

Tiêu chí	Permanent Tablespace	Temporary Tablespace	Undo Tablespace
Mục đích chính	Lưu trữ dữ liệu lâu dài	Xử lý dữ liệu tạm	Quản lý giao dịch

Tính chất dữ liệu	Bền vững	Tạm thời	Trung gian (lịch sử)
Vai trò hệ thống	Trung tâm dữ liệu	Hỗ trợ truy vấn	Đảm bảo toàn vẹn
Liên quan ACID	Durability, Consistency	Không trực tiếp	Atomicity, Consistency
Ảnh hưởng hiệu năng	Truy xuất dữ liệu	Tối ưu query	Tăng concurrency
Khả năng phục hồi	Có	Không cần	Có (ROLLBACK, FLASHBACK)

Bảng 2.1: So sánh tổng hợp Tablespace trong Oracle

Tổng thể, ba loại Tablespace trong Oracle tạo thành một kiến trúc lưu trữ và xử lý dữ liệu đa tầng, trong đó mỗi loại đảm nhận một vai trò riêng biệt nhưng có mối liên hệ chặt chẽ với nhau. Permanent Tablespace đảm nhiệm lưu trữ dữ liệu lâu dài, Temporary Tablespace hỗ trợ xử lý truy vấn, và Undo Tablespace đảm bảo tính toàn vẹn giao dịch.

Sự kết hợp này không chỉ giúp hệ thống hoạt động hiệu quả mà còn đảm bảo khả năng mở rộng, tính ổn định và độ tin cậy cao trong các môi trường thực tế. Việc hiểu rõ và tối ưu hóa từng loại Tablespace là một trong những nhiệm vụ quan trọng nhất của quản trị viên cơ sở dữ liệu, đặc biệt trong bối cảnh các hệ thống ngày càng phức tạp và yêu cầu hiệu năng ngày càng cao.

2.3. Cơ chế quản lý dung lượng và Quota:

Trong Oracle 19c, việc quản lý dung lượng lưu trữ được thực hiện thông qua sự kết hợp giữa cơ chế kiểm soát kích thước file vật lý (datafile) và hệ thống phân bổ hạn mức dung lượng (quota) cho từng người dùng. Hai cơ chế này hoạt động song song nhằm đảm bảo rằng tài nguyên lưu trữ được sử dụng hiệu quả, đồng thời duy trì tính ổn định và kiểm soát trong môi trường đa người dùng.

2.3.1. Cơ chế kiểm soát kích thước file vật lý:

Trong kiến trúc của Oracle, dữ liệu được lưu trữ trong các Tablespace, nhưng bản thân Tablespace lại được ánh xạ xuống các file vật lý gọi là datafile. Do đó, dung lượng lưu trữ thực tế của hệ thống phụ thuộc trực tiếp vào kích thước và cách quản lý các datafile này.

Oracle cung cấp hai cơ chế chính để kiểm soát kích thước datafile: cấp phát thủ công và tự động mở rộng (autoextend). Trong cơ chế thủ công, quản trị viên phải chủ động điều chỉnh kích thước datafile thông qua các lệnh quản trị khi dung lượng gần đạt giới hạn. Ngược lại, với cơ chế autoextend, hệ thống có thể tự động tăng kích thước datafile theo một mức xác định khi không gian lưu trữ gần đầy. Cơ chế này giúp giảm thiểu nguy cơ gián đoạn hệ thống do thiếu dung lượng, đồng thời nâng cao tính linh hoạt trong vận hành.

Bên cạnh đó, mỗi datafile có thể được thiết lập giới hạn kích thước tối đa (maxsize), nhằm kiểm soát mức độ sử dụng tài nguyên lưu trữ. Điều này đặc biệt quan trọng trong các hệ thống có tài nguyên hạn chế hoặc yêu cầu phân bổ dung lượng rõ ràng giữa các thành phần.

Ở mức độ nội tại, Oracle quản lý không gian trong datafile thông qua các đơn vị như block, extent và segment. Khi một đối tượng dữ liệu được tạo ra hoặc mở rộng, hệ thống sẽ cấp phát các extent từ vùng không gian còn trống trong datafile. Cơ chế này giúp tối ưu hóa việc sử dụng dung lượng và giảm thiểu phân mảnh.

Ngoài ra, Oracle còn cung cấp các công cụ giám sát như các view hệ thống (ví dụ: DBA_DATA_FILES, DBA_FREE_SPACE) cho phép quản trị viên theo dõi tình trạng sử dụng dung lượng theo thời gian thực. Nhờ đó, việc quản lý dung lượng không chỉ mang tính phản ứng mà còn có thể mang tính chủ động, giúp dự đoán và xử lý các vấn đề trước khi xảy ra.

2.3.2. Cơ chế thiết lập và kiểm soát Quota:

Bên cạnh việc kiểm soát dung lượng ở mức hệ thống, Oracle còn cung cấp cơ chế quota nhằm quản lý việc sử dụng tài nguyên ở cấp độ người dùng. Quota cho

phép giới hạn dung lượng mà mỗi user có thể sử dụng trên một Tablespace cụ thể, từ đó đảm bảo sự phân bổ tài nguyên hợp lý trong môi trường đa người dùng.

Cơ chế quota được thiết lập riêng biệt cho từng user và từng Tablespace, cho phép quản trị viên kiểm soát chi tiết mức độ sử dụng tài nguyên. Mỗi user có thể được cấp một mức quota xác định hoặc không giới hạn (UNLIMITED), tùy thuộc vào vai trò và nhu cầu sử dụng trong hệ thống.

Khi một user thực hiện thao tác tạo hoặc mở rộng đối tượng dữ liệu, hệ thống sẽ kiểm tra quota trước khi cấp phát thêm không gian. Nếu dung lượng sử dụng vượt quá quota đã được thiết lập, thao tác sẽ bị từ chối. Cơ chế này đóng vai trò như một lớp kiểm soát nhằm ngăn chặn việc sử dụng tài nguyên quá mức, đồng thời bảo vệ hệ thống khỏi các tình trạng quá tải hoặc thiếu dung lượng.

Ngoài ra, quota còn góp phần hỗ trợ quản lý bảo mật và phân quyền trong hệ thống. Việc giới hạn dung lượng không chỉ mang ý nghĩa kỹ thuật mà còn phản ánh chính sách quản trị, giúp phân chia tài nguyên theo vai trò của từng nhóm người dùng.

Tương tự như quản lý datafile, Oracle cũng cung cấp các công cụ giám sát quota thông qua các view hệ thống như DBA_TS_QUOTAS. Nhờ đó, quản trị viên có thể theo dõi mức sử dụng dung lượng của từng user và điều chỉnh quota khi cần thiết, đảm bảo hệ thống luôn hoạt động trong trạng thái tối ưu.

Sự kết hợp giữa hai cơ chế này không chỉ giúp tối ưu hóa hiệu năng hệ thống mà còn đảm bảo tính công bằng, an toàn và khả năng kiểm soát trong môi trường đa người dùng. Đây là một trong những yếu tố quan trọng giúp Oracle duy trì hiệu quả hoạt động và độ tin cậy cao trong các hệ thống cơ sở dữ liệu quy mô lớn.

CHƯƠNG 3. QUẢN LÝ GIAO DỊCH (TRANSACTION) VÀ CƠ CHẾ ĐẢM BẢO TÍNH TOÀN VỆ DỮ LIỆU TRONG ORACLE

3.1. Giao dịch như một đơn vị logic trong đảm bảo toàn vẹn dữ liệu:

Trong các hệ quản trị cơ sở dữ liệu hiện đại, đặc biệt là trong những hệ thống có cường độ truy cập cao và yêu cầu độ tin cậy nghiêm ngặt, việc duy trì tính toàn vẹn của dữ liệu không thể tách rời khỏi cơ chế quản lý giao dịch. Trong Oracle, TRANSACTION không chỉ được xem là một tập hợp các thao tác DML, mà còn là một thực thể logic đóng vai trò trung tâm trong việc điều phối, kiểm soát và bảo vệ trạng thái dữ liệu.

Về phương diện lý thuyết, TRANSACTION được xây dựng dựa trên mô hình ACID, trong đó bốn thuộc tính Atomicity, Consistency, Isolation và Durability không tồn tại độc lập mà tương tác chặt chẽ nhằm đảm bảo rằng mọi thay đổi đối với dữ liệu đều diễn ra trong một khuôn khổ kiểm soát nghiêm ngặt. Tuy nhiên, điểm đáng chú ý trong Oracle không nằm ở việc tuân thủ ACID – vốn là tiêu chuẩn chung – mà ở cách hệ thống hiện thực hóa các nguyên lý này thông qua các cơ chế nội tại như Undo, Redo và MVCC.

Một đặc trưng quan trọng trong Oracle là việc khởi tạo TRANSACTION mang tính ngầm định. Không giống như nhiều hệ quản trị khác yêu cầu khai báo tường minh điểm bắt đầu, Oracle tự động khởi tạo TRANSACTION ngay khi câu lệnh DML đầu tiên được thực thi. Điều này không chỉ đơn giản hóa mô hình lập trình mà còn phản ánh triết lý thiết kế hướng đến tính tự động hóa và giảm thiểu sai sót do con người.

Xét về vòng đời, một TRANSACTION có thể được hiểu như một tiến trình chuyển đổi trạng thái dữ liệu qua các pha liên tiếp: khởi tạo, thực thi, ghi nhận và kết thúc. Trong pha thực thi, dữ liệu không được cập nhật trực tiếp lên bộ nhớ lưu trữ lâu dài, mà được xử lý thông qua buffer cache, đồng thời các thông tin liên quan đến khả năng phục hồi được ghi nhận song song. Cách tiếp cận này cho phép Oracle đạt được sự cân bằng giữa hiệu năng và độ an toàn: giảm thiểu chi phí I/O nhưng vẫn đảm bảo khả năng khôi phục trong mọi tình huống.

Điểm kết thúc của TRANSACTION – thông qua COMMIT hoặc ROLLBACK – không chỉ mang ý nghĩa kỹ thuật, mà còn là ranh giới logic phân tách giữa các trạng thái nhất quán của hệ thống. Một TRANSACTION chỉ thực sự tồn tại có ý nghĩa khi nó có thể đảm bảo rằng hệ thống chuyển từ một trạng thái hợp lệ sang một trạng thái hợp lệ khác, bất kể quá trình trung gian có thể bị gián đoạn bởi lỗi hay xung đột.

3.2. Cơ chế Undo và Redo - Hai trụ cột của tính nhất quán và khả năng phục hồi:

Để hiện thực hóa các nguyên lý ACID trong môi trường vận hành thực tế, Oracle triển khai một kiến trúc lưu trữ trung gian dựa trên hai thành phần cốt lõi: Undo và Redo Log. Hai cơ chế này không chỉ đóng vai trò hỗ trợ TRANSACTION, mà còn là nền tảng cho toàn bộ hệ thống đảm bảo tính nhất quán và phục hồi dữ liệu.

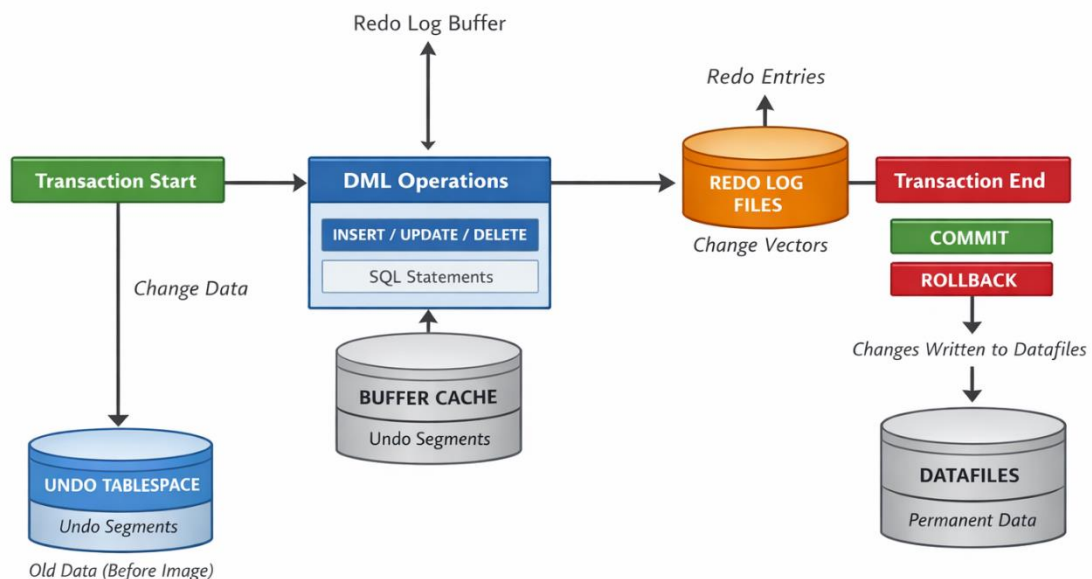


Figure 1: Transaction Processing in Oracle – Undo and Redo Mechanisms

Hình 3.1: Quy trình xử lý giao dịch trong Oracle với cơ chế Undo và Redo.

(Nguồn: Nhóm sinh viên tự thực hiện)

Trước hết, Undo có thể được xem như một lớp “lịch sử dữ liệu”, trong đó lưu trữ các phiên bản trước của dữ liệu mỗi khi xảy ra thay đổi. Khác với cách tiếp cận ghi đè

trực tiếp, Oracle sử dụng chiến lược “copy-before-write”, theo đó dữ liệu gốc được lưu vào Undo trước khi áp dụng thay đổi mới. Điều này cho phép hệ thống duy trì khả năng truy vết ngược về trạng thái trước đó của dữ liệu – một yếu tố then chốt trong việc đảm bảo Atomicity.

Tuy nhiên, vai trò của Undo không dừng lại ở việc hỗ trợ ROLLBACK. Trong môi trường đa người dùng, nơi các TRANSACTION diễn ra đồng thời, Undo trở thành nền tảng cho cơ chế đọc nhất quán. Thay vì buộc các truy vấn đọc phải chờ các thao tác ghi hoàn tất, Oracle sử dụng dữ liệu trong Undo để tái tạo lại trạng thái dữ liệu tại thời điểm truy vấn bắt đầu. Cơ chế này cho phép các TRANSACTION đọc và ghi cùng tồn tại mà không gây xung đột trực tiếp, từ đó nâng cao đáng kể hiệu năng hệ thống.

Song song với Undo, Redo Log đóng vai trò như một cơ chế ghi nhận các thay đổi theo hướng tiến (forward recovery). Thay vì lưu trữ dữ liệu hoàn chỉnh, Redo chỉ ghi lại các thay đổi dưới dạng các vector thao tác. Điều này giúp giảm thiểu dung lượng lưu trữ và tăng tốc độ ghi log, đồng thời vẫn đảm bảo rằng hệ thống có thể tái tạo lại toàn bộ trạng thái dữ liệu sau khi xảy ra sự cố.

Một nguyên lý quan trọng chi phối hoạt động của Redo là “write-ahead logging”, theo đó mọi thay đổi phải được ghi nhận vào Redo Log trước khi được ghi xuống datafile. Khi COMMIT được thực thi, tiến trình LGWR sẽ đảm bảo rằng toàn bộ Redo Log liên quan đã được ghi an toàn xuống đĩa. Chỉ khi điều kiện này được thỏa mãn, TRANSACTION mới được coi là hoàn tất. Điều này đảm bảo rằng, ngay cả khi hệ thống gặp sự cố ngay sau thời điểm COMMIT, dữ liệu vẫn có thể được phục hồi đầy đủ thông qua việc phát lại các log.

Sự kết hợp giữa Undo và Redo tạo nên một cơ chế bảo vệ dữ liệu hai chiều: Undo cho phép quay ngược về quá khứ, trong khi Redo đảm bảo khả năng tiến về trạng thái tương lai. Chính sự hỗ trợ này đã giúp Oracle đạt được mức độ tin cậy cao trong cả xử lý giao dịch lẫn phục hồi sau sự cố.

3.3. Điều khiển giao dịch - Cơ chế can thiệp vào trạng thái nhất quán:

Trong khi các cơ chế nội tại như Undo và Redo hoạt động một cách tự động, Oracle vẫn cung cấp cho người dùng các công cụ để chủ động kiểm soát TRANSACTION. Các lệnh như COMMIT, ROLLBACK và SAVEPOINT không chỉ là các thao tác cú pháp, mà còn là các điểm can thiệp trực tiếp vào trạng thái logic của hệ thống.

COMMIT đánh dấu sự chấp nhận toàn bộ thay đổi trong một TRANSACTION, đồng thời chuyển các thay đổi này từ trạng thái tạm thời sang trạng thái vĩnh viễn. Tại thời điểm này, các khóa liên quan được giải phóng, mở đường cho các TRANSACTION khác truy cập dữ liệu. Quan trọng hơn, COMMIT kích hoạt quá trình đồng bộ hóa giữa bộ nhớ và đĩa thông qua Redo Log, qua đó đảm bảo tính bền vững của dữ liệu.

Ngược lại, ROLLBACK cho phép hệ thống hủy bỏ các thay đổi chưa được xác nhận. Dựa trên dữ liệu Undo, Oracle có thể phục hồi chính xác trạng thái dữ liệu trước khi TRANSACTION bắt đầu. Cơ chế này đóng vai trò như một “hệ thống an toàn nội tại”, cho phép xử lý các lỗi logic hoặc các tình huống bất thường mà không làm ảnh hưởng đến toàn bộ hệ thống.

SAVEPOINT cung cấp một mức độ kiểm soát tinh vi hơn, cho phép thiết lập các điểm trung gian trong TRANSACTION. Thay vì phải hủy bỏ toàn bộ, người dùng có thể quay lại một trạng thái cụ thể đã được đánh dấu trước đó. Điều này đặc biệt hữu ích trong các quy trình xử lý phức tạp, nơi việc ROLLBACK toàn phần có thể gây lãng phí tài nguyên và thời gian.

Thông qua các cơ chế điều khiển này, Oracle không chỉ đảm bảo tính toàn vẹn dữ liệu một cách tự động, mà còn trao quyền cho người dùng trong việc quản lý và tối ưu hóa hành vi giao dịch.

3.4. Tính cô lập và nhất quán trong môi trường đa giao dịch:

Một trong những thách thức cốt lõi của hệ quản trị cơ sở dữ liệu là đảm bảo rằng các TRANSACTION đồng thời không gây ra các trạng thái dữ liệu không nhất quán. Oracle tiếp cận vấn đề này thông qua sự kết hợp giữa các mức độ cô lập và cơ chế MVCC.

Ở mức mặc định, Oracle sử dụng chế độ read committed, trong đó mỗi truy vấn chỉ quan sát dữ liệu đã được COMMIT tại thời điểm bắt đầu. Điều này giúp tránh hiện tượng đọc dữ liệu chưa xác nhận (dirty read), đồng thời duy trì hiệu năng cao do không cần khóa dữ liệu trong suốt TRANSACTION.

Trong các trường hợp yêu cầu mức độ cô lập cao hơn, Oracle cung cấp chế độ serializable, cho phép các TRANSACTION hoạt động như thể chúng được thực thi tuần tự. Tuy nhiên, mức độ này có thể dẫn đến xung đột và yêu cầu ROLLBACK nếu hệ thống phát hiện sự vi phạm tính tuần tự.

Điểm khác biệt của Oracle so với nhiều hệ quản trị khác nằm ở việc giảm thiểu sự phụ thuộc vào locking. Thay vì sử dụng khóa để ngăn chặn xung đột, Oracle tận dụng Undo để cung cấp các phiên bản dữ liệu khác nhau cho từng TRANSACTION. Cách tiếp cận này không chỉ giảm thiểu hiện tượng blocking mà còn nâng cao khả năng mở rộng của hệ thống trong môi trường nhiều người dùng.

3.5. Đánh giá và hàm ý thực tiễn:

Từ góc độ kiến trúc hệ thống, có thể nhận thấy rằng TRANSACTION trong Oracle không chỉ là một cơ chế xử lý dữ liệu, mà còn là một khung điều phối toàn diện, kết nối các thành phần khác nhau như bộ nhớ, log và cơ chế đồng thời. Sự phối hợp nhịp nhàng giữa Undo, Redo và các lệnh điều khiển TRANSACTION cho phép Oracle đạt được một sự cân bằng hiếm có giữa hiệu năng, độ tin cậy và khả năng mở rộng.

Về mặt thực tiễn, việc hiểu rõ cơ chế TRANSACTION giúp các nhà phát triển và quản trị viên cơ sở dữ liệu đưa ra các quyết định thiết kế hợp lý, chẳng hạn như:

- Xác định thời điểm COMMIT tối ưu để giảm thiểu lock và tránh fragment TRANSACTION;
- Sử dụng SAVEPOINT trong các quy trình phức tạp;
- Thiết kế ứng dụng phù hợp với cơ chế MVCC để tận dụng tối đa hiệu năng.

Những hiểu biết này không chỉ giúp tối ưu hóa hệ thống hiện tại, mà còn là nền tảng để xây dựng các ứng dụng có khả năng mở rộng và chịu lỗi cao trong tương lai.

CHƯƠNG 4. CƠ CHẾ KHÓA (LOCK) VÀ ĐIỀU KHIỂN ĐỒNG THỜI

4.1. Khái niệm và mục đích của Lock:

Trong các hệ quản trị cơ sở dữ liệu cấp doanh nghiệp, số lượng người dùng truy cập và thao tác trên hệ thống tại cùng một thời điểm có thể lên đến hàng nghìn, thậm chí hàng chục nghìn. Các thao tác như truy vấn, cập nhật, thêm hoặc xóa dữ liệu diễn ra đồng thời trên cùng một tập dữ liệu, tạo nên môi trường xử lý đồng thời (concurrency). Đây là một đặc trưng quan trọng giúp hệ thống nâng cao hiệu suất và khả năng phục vụ nhiều người dùng cùng lúc. Tuy nhiên, chính sự đồng thời này lại tiềm ẩn nhiều rủi ro nghiêm trọng nếu không được kiểm soát một cách chặt chẽ.

Cụ thể, khi nhiều giao dịch cùng truy cập và thao tác trên cùng một bản ghi dữ liệu mà không có cơ chế điều phối hợp lý, các hiện tượng bất thường có thể phát sinh. Một trong những trường hợp điển hình là hiện tượng “cập nhật bị mất” (lost update), xảy ra khi hai hoặc nhiều giao dịch cùng sửa đổi một giá trị dữ liệu và kết quả cập nhật của một giao dịch bị ghi đè bởi giao dịch khác mà không được phát hiện. Bên cạnh đó, hiện tượng “đọc dữ liệu chưa được xác nhận” (dirty read) cũng là một vấn đề nghiêm trọng, khi một giao dịch đọc phải dữ liệu đang được thay đổi bởi một giao dịch khác nhưng chưa hoàn tất, dẫn đến việc sử dụng thông tin không chính xác hoặc không nhất quán.

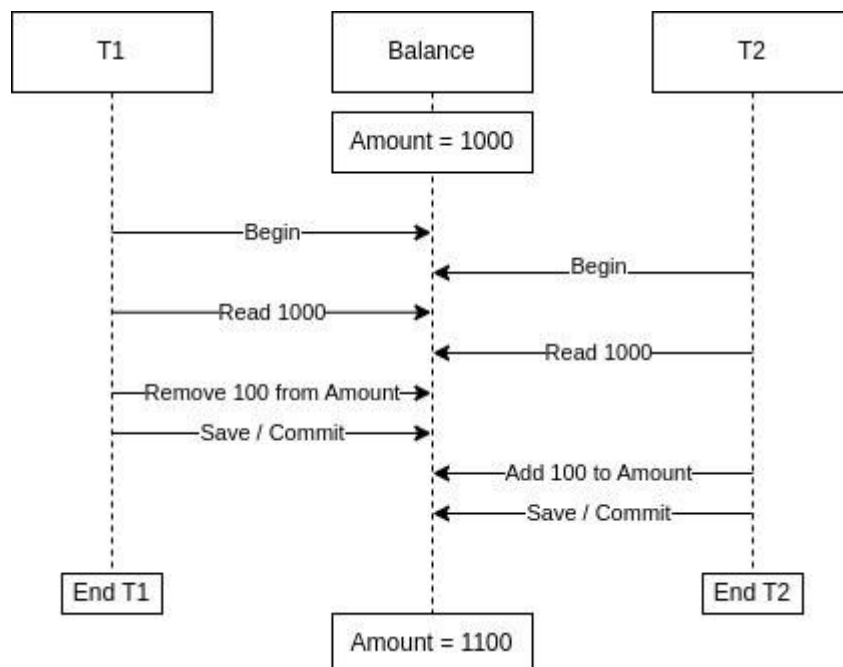
Để hạn chế và ngăn chặn các dị thường này, hệ quản trị cơ sở dữ liệu Oracle triển khai cơ chế khóa (lock) như một phương thức kiểm soát truy cập vào dữ liệu. Về bản chất, lock là một công cụ cho phép hệ thống kiểm soát quyền thao tác của các giao dịch đối với một đối tượng dữ liệu cụ thể tại từng thời điểm. Khi một giao dịch bắt đầu thực hiện thao tác làm thay đổi dữ liệu, hệ thống sẽ tự động thiết lập một khóa tương ứng lên phần dữ liệu đó nhằm đảm bảo rằng các giao dịch khác không thể thực hiện các thao tác xung đột trong cùng thời điểm.

Mục tiêu cốt lõi của cơ chế khóa là đảm bảo rằng các giao dịch được thực thi một cách độc lập và có kiểm soát, từ đó duy trì tính cô lập và tính nhất quán của dữ liệu trong toàn hệ thống. Khi một bản ghi đã bị khóa bởi một giao dịch, các giao dịch khác nếu

muốn thực hiện thao tác ghi lên cùng bản ghi đó sẽ không được phép thực hiện ngay lập tức, mà phải chuyển sang trạng thái chờ. Trong thời gian này, các giao dịch bị trì hoãn sẽ được xếp vào một hàng đợi chờ (wait queue) và chỉ được tiếp tục xử lý khi khóa được giải phóng.

Việc giải phóng khóa xảy ra khi giao dịch đang nắm giữ khóa kết thúc, thông qua một trong hai thao tác là COMMIT hoặc ROLLBACK. Nếu giao dịch được COMMIT, các thay đổi sẽ được ghi nhận chính thức và khóa được mở để các giao dịch khác tiếp tục thao tác. Ngược lại, nếu giao dịch bị ROLLBACK, hệ thống sẽ phục hồi dữ liệu về trạng thái ban đầu trước khi giao dịch diễn ra, đồng thời giải phóng khóa để đảm bảo các giao dịch khác không bị đình trệ.

Nhờ cơ chế này, các thao tác cập nhật dữ liệu được thực hiện theo một trình tự có kiểm soát, tránh tình trạng nhiều giao dịch cùng ghi đè lên một dữ liệu tại cùng một thời điểm. Điều này giúp đảm bảo rằng mọi thay đổi đối với dữ liệu đều diễn ra một cách chính xác, nhất quán và có thể dự đoán được, từ đó duy trì độ tin cậy và tính toàn vẹn của hệ thống cơ sở dữ liệu trong môi trường xử lý đồng thời.



Hình 4.1: Khái niệm và mục đích của Lock.

(Nguồn: Baeldung, [“The Lost Update Problem in Concurrency Control”](#), 2024)

4.2. Phân loại cơ chế Lock:

Trong môi trường xử lý đồng thời, việc triển khai cơ chế khóa không chỉ nhằm đảm bảo tính toàn vẹn của thông tin mà còn phải duy trì hiệu năng hoạt động của toàn hệ thống. Nếu phạm vi kiểm soát quá rộng, các tiến trình sẽ bị đình trệ do phải chờ đợi lẫn nhau; ngược lại, nếu kiểm soát quá lỏng lẻo, nguy cơ phát sinh xung đột và sai lệch thông tin sẽ tăng cao. Chính vì vậy, Oracle thiết kế nhiều cấp độ kiểm soát khác nhau để điều phối truy cập một cách linh hoạt và hiệu quả.

Đối với các thao tác xử lý dữ liệu (DML), hai hình thức phổ biến nhất là khóa ở mức dòng và khóa ở mức bảng. Hai cơ chế này không tồn tại độc lập mà thường phối hợp với nhau trong cùng một thao tác, mỗi loại đảm nhiệm một vai trò riêng biệt trong việc bảo vệ hệ thống.

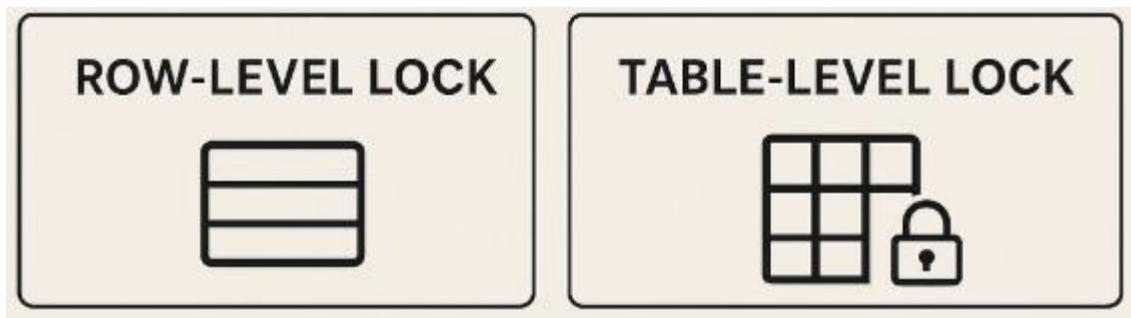
Trước hết, khóa cấp dòng (Row-level Lock – TX Lock) là hình thức kiểm soát có độ chi tiết cao nhất, tác động trực tiếp đến từng bản ghi cụ thể. Khi một thao tác như update hoặc delete được thực hiện, hệ thống chỉ áp dụng cơ chế kiểm soát lên những dòng bị ảnh hưởng, thay vì áp dụng trên toàn bộ bảng. Nhờ đó, các tiến trình khác vẫn có thể truy cập hoặc chỉnh sửa những phần thông tin không liên quan mà không bị gián đoạn.

Ví dụ, khi một người dùng thay đổi thông tin của một sinh viên trong bảng “SinhVien”, chỉ bản ghi tương ứng bị đặt dưới sự kiểm soát. Các người dùng khác vẫn có thể thao tác trên những sinh viên còn lại mà không gặp trở ngại. Cách tiếp cận này giúp nâng cao khả năng xử lý song song, đồng thời hạn chế tối đa tình trạng tắc nghẽn trong hệ thống.

Bên cạnh đó, Oracle còn áp dụng khóa cấp bảng (Table-level Lock – TM Lock) như một cơ chế bảo vệ ở phạm vi rộng hơn. Khi một thao tác thay đổi nội dung được thực hiện, ngoài việc kiểm soát ở mức dòng, hệ thống sẽ đồng thời thiết lập một cơ chế kiểm soát trên toàn bộ bảng. Tuy nhiên, khác với suy nghĩ thông thường, cơ chế này không nhằm ngăn cản các thao tác cập nhật dữ liệu từ các tiến trình khác.

Thực chất, vai trò chính của TM Lock là ngăn chặn các thay đổi liên quan đến cấu trúc bảng trong quá trình dữ liệu đang được xử lý. Điều này có nghĩa là các lệnh như drop table, alter table hoặc truncate table sẽ không thể thực hiện khi bảng đang được sử dụng. Cơ chế này giúp duy trì sự ổn định của cấu trúc hệ thống, tránh những thay đổi đột ngột có thể gây lỗi hoặc làm gián đoạn quá trình xử lý.

Nhìn chung, hai cấp độ kiểm soát này bổ trợ lẫn nhau nhằm đảm bảo an toàn cho hệ thống ở cả hai phương diện: nội dung và cấu trúc. Việc giới hạn phạm vi tác động ở mức dòng giúp tối ưu khả năng xử lý đồng thời, trong khi cơ chế ở mức bảng đóng vai trò như một lớp bảo vệ nền tảng, đảm bảo môi trường hoạt động luôn ổn định và nhất quán.



Hình 4.2: Phạm vi tác động của khóa dòng và khóa bảng.

(Nguồn: Nhóm sinh viên thực hiện)

4.3. Hiện tượng chờ khóa và khóa chết:

Sự cạnh tranh trong việc truy cập tài nguyên là đặc điểm tất yếu của các hệ thống cơ sở dữ liệu có nhiều người dùng hoạt động đồng thời. Khi nhiều phiên làm việc cùng yêu cầu thao tác trên một đối tượng, hệ thống buộc phải điều phối các yêu cầu này để tránh xung đột và đảm bảo dữ liệu không bị sai lệch. Trong quá trình đó, hai trạng thái thường xuyên xuất hiện là chờ khóa (blocking) và khóa chết (deadlock), mỗi trạng thái phản ánh một mức độ xung đột khác nhau.

Chờ khóa (blocking) là cơ chế kiểm soát được thiết kế sẵn nhằm điều phối quyền truy cập vào tài nguyên. Trạng thái này xuất hiện khi một phiên đang nắm quyền thao tác trên một đối tượng, trong khi phiên khác yêu cầu quyền truy cập không tương thích lên cùng đối tượng đó. Thay vì cho phép truy cập đồng thời gây xung đột, hệ thống sẽ tạm hoãn yêu cầu đến sau.

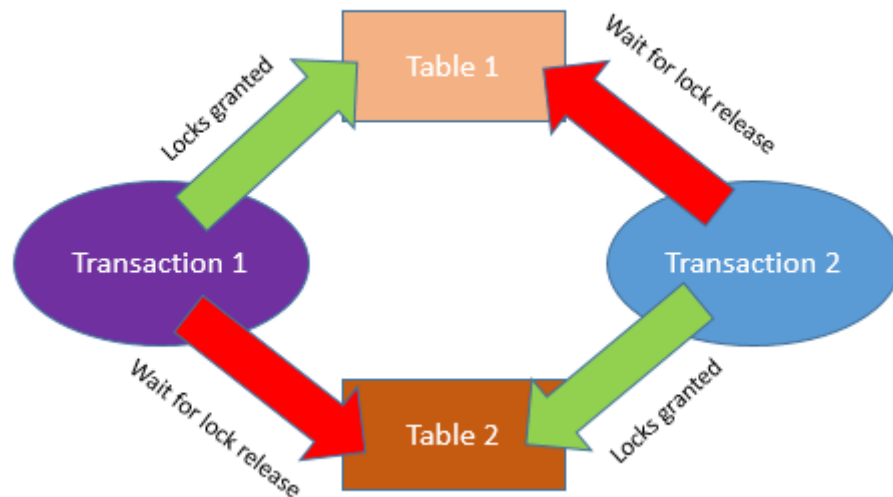
Xét một tình huống cụ thể: nếu một phiên đang thực hiện cập nhật trên một bản ghi và chưa hoàn tất, mọi yêu cầu chỉnh sửa bản ghi đó từ các phiên khác sẽ bị trì hoãn. Các yêu cầu này được đưa vào hàng đợi và chỉ được xử lý khi phiên đang giữ quyền thao tác kết thúc bằng COMMIT hoặc ROLLBACK. Cách điều phối này giúp đảm bảo các thay đổi được thực hiện theo trình tự rõ ràng, tránh hiện tượng ghi đè lẫn nhau. Vì vậy, blocking không phải là dấu hiệu bất thường mà là một cơ chế cần thiết để duy trì sự ổn định của hệ thống.

Trái lại, khóa chết (deadlock) phản ánh một tình huống bế tắc trong quá trình điều phối tài nguyên. Hiện tượng này xảy ra khi các phiên làm việc phụ thuộc lẫn nhau theo một chuỗi khép kín, trong đó mỗi bên đều đang chờ tài nguyên do bên khác nắm giữ. Kết quả là toàn bộ các phiên liên quan đều bị “đóng băng” và không thể tiếp tục xử lý.

Một ví dụ điển hình là khi phiên A giữ quyền trên bản ghi thứ nhất và yêu cầu bản ghi thứ hai, trong khi phiên B lại giữ bản ghi thứ hai và yêu cầu bản ghi thứ nhất. Sự phụ thuộc chéo này tạo thành một vòng lặp mà không bên nào có thể tự thoát ra. Không giống như blocking, trạng thái này không thể tự kết thúc nếu không có sự can thiệp từ hệ thống.

Để xử lý tình huống này, Oracle tích hợp cơ chế phát hiện deadlock tự động. Khi nhận diện được vòng lặp chờ giữa các phiên, hệ thống sẽ lựa chọn một tiến trình để hủy bỏ thao tác hiện tại nhằm phá vỡ trạng thái bế tắc. Phiên bị dừng sẽ nhận thông báo lỗi (thường là ORA-00060), trong khi các phiên còn lại được giải phóng và tiếp tục thực thi. Cách tiếp cận này giúp hạn chế tối đa ảnh hưởng của deadlock đến toàn bộ hệ thống.

Sự khác biệt cốt lõi giữa hai hiện tượng nằm ở khả năng tự giải quyết. Nếu blocking chỉ là trạng thái chờ tạm thời và sẽ được xử lý khi tài nguyên được giải phóng, thì deadlock lại là một dạng xung đột logic đòi hỏi hệ thống phải chủ động can thiệp. Việc nhận diện và phân biệt chính xác hai trường hợp này có ý nghĩa quan trọng trong quá trình vận hành cũng như tối ưu hiệu năng của hệ quản trị cơ sở dữ liệu.



Hình 4.3: Sơ đồ minh họa trạng thái chờ đợi vòng tròn dẫn đến khóa chết.

(Nguồn: [Viblo](#), “*Deadlock trong SQL transaction - Ứng dụng trong Laravel*”, 2020)

CHƯƠNG 5. TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ

Nhằm kiểm chứng các cơ sở lý thuyết đã trình bày ở các chương trước, phần thực nghiệm được triển khai trên hệ quản trị cơ sở dữ liệu Oracle 19c trong môi trường Linux. Nội dung thực nghiệm tập trung vào ba khía cạnh chính: quản trị không gian lưu trữ (Tablespace), kiểm soát giao dịch (TRANSACTION) và xử lý đồng thời (Concurrency Control). Các kịch bản được xây dựng theo hướng mô phỏng các tình huống thực tế nhằm đánh giá khả năng vận hành và đảm bảo tính toàn vẹn dữ liệu của hệ thống.

5.1. Thực nghiệm quản trị Tablespace:

Để đảm bảo hệ thống cơ sở dữ liệu hoạt động ổn định và có khả năng mở rộng, việc tách biệt giữa cấu trúc logic (Tablespace) và cấu trúc vật lý (Datafile) là yêu cầu quan trọng. Nội dung thực nghiệm dưới đây minh họa quá trình cấp phát và quản lý không gian lưu trữ.

5.1.1. Khởi tạo Tablespace và định tuyến tệp vật lý:

5.1.1.1. Cú pháp lệnh:

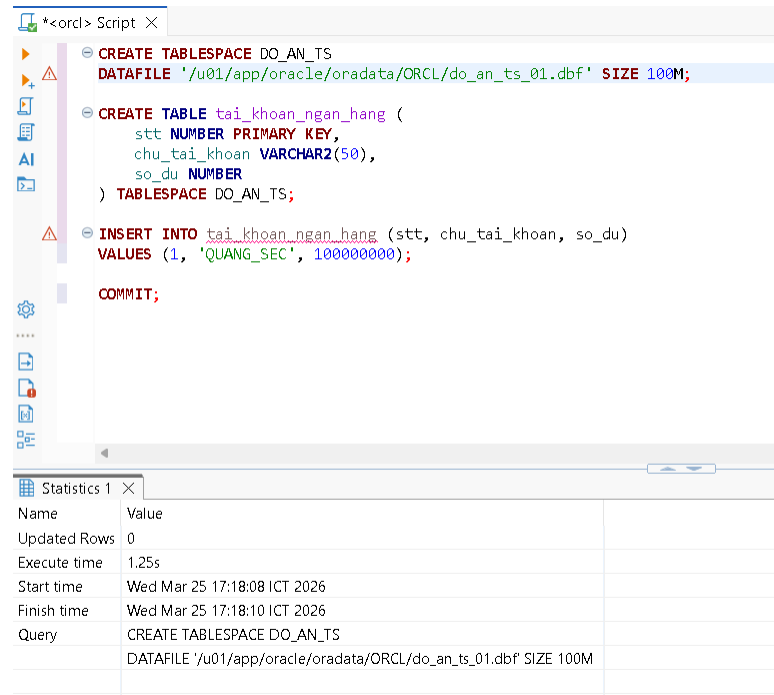
```
CREATE TABLESPACE <tên_tablespace>  
  
DATAFILE '<đường_dẫn_lưu_tệp_vật_lý>' SIZE <kích_thuớc_ban_đầu>;
```

5.1.1.2. Mô tả và thực thi:

Nhóm tiến hành tạo một Permanent Tablespace với tên DO_AN_TS, được ánh xạ trực tiếp xuống hệ thống tệp của hệ điều hành thông qua Datafile do_an_ts_01.dbf với dung lượng ban đầu là 100MB.

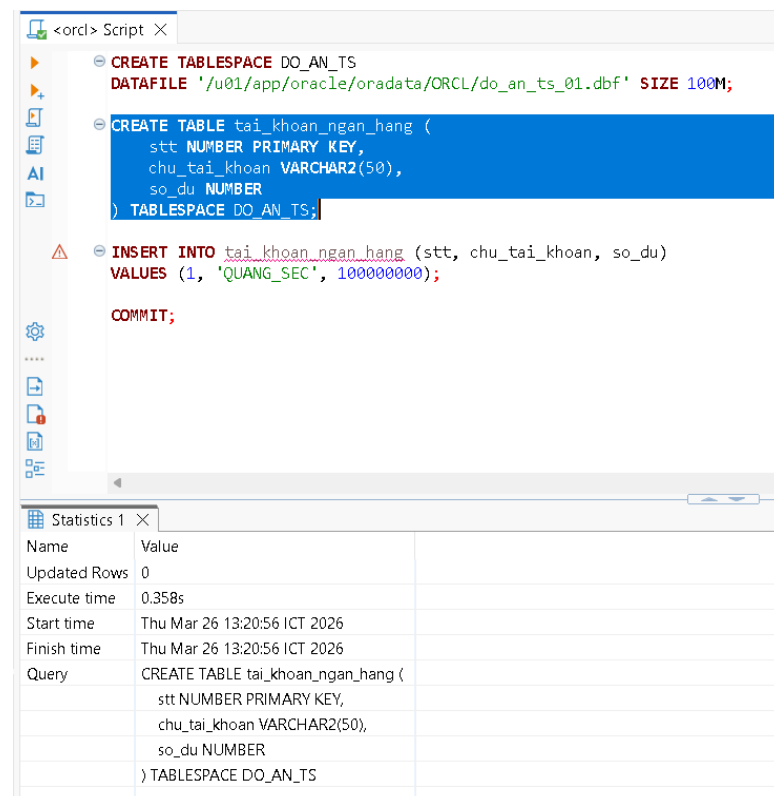
```
CREATE TABLESPACE DO_AN_TS  
  
DATAFILE '/u01/app/oracle/oradata/ORCL/do_an_ts_01.dbf' SIZE 100M
```

Sau đó, bảng *tai_khoan_ngan_hang* được khởi tạo và chỉ định lưu trữ trên Tablespace DO_AN_TS. Việc này giúp dữ liệu được tổ chức tập trung và dễ kiểm soát trong quá trình vận hành.



Hình 5.1: Kết quả thực thi lệnh Create Tablespace.

(Nguồn: Nhóm sinh viên thực hiện)



Hình 5.2: Kết quả thực thi lệnh Create Table.

(Nguồn: Nhóm sinh viên thực hiện)

5.1.2. Mở rộng dung lượng Datafile và cấp quyền Quota:

5.1.2.1. Cú pháp lệnh:

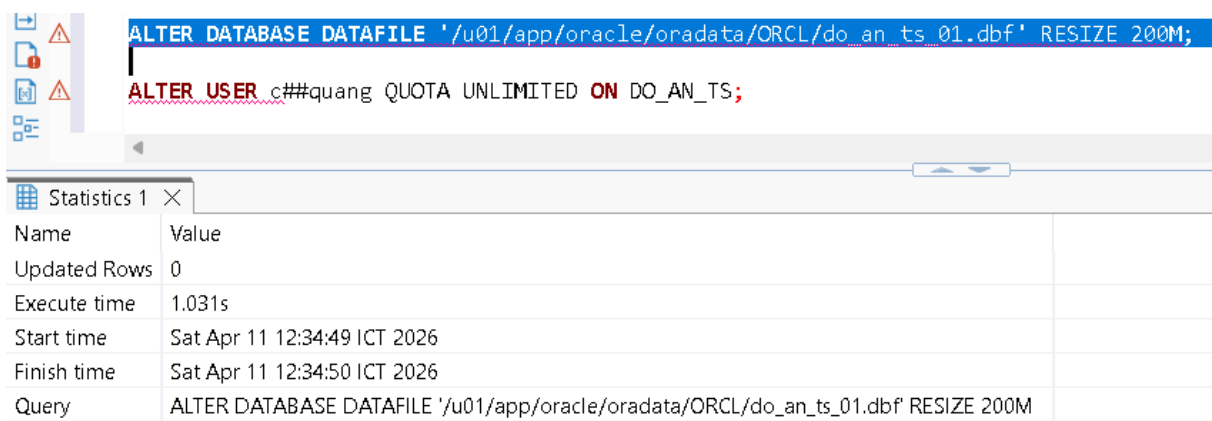
```
ALTER DATABASE DATAFILE '<đường_dẫn_tệp>' RESIZE <kích_thuớc_mới>;  
  
ALTER USER <tên_người_dùng> QUOTA <hạn_mức> ON <tên_tablespace  
  
>;
```

5.1.2.2. Mô tả và thực thi:

Khi dung lượng lưu trữ ban đầu không còn đáp ứng nhu cầu, quản trị viên có thể mở rộng Datafile mà không cần dừng hệ thống. Trong thực nghiệm, Datafile được tăng từ 100MB lên 200MB, đồng thời cấp quyền QUOTA UNLIMITED cho người dùng c##quang trên Tablespace DO_AN_T.

```
ALTER DATABASE DATAFILE  
'/u01/app/oracle/oradata/ORCL/do_an_ts_01.dbf' RESIZE 200M;  
  
ALTER USER c##quang QUOTA UNLIMITED ON DO_AN_TS;
```

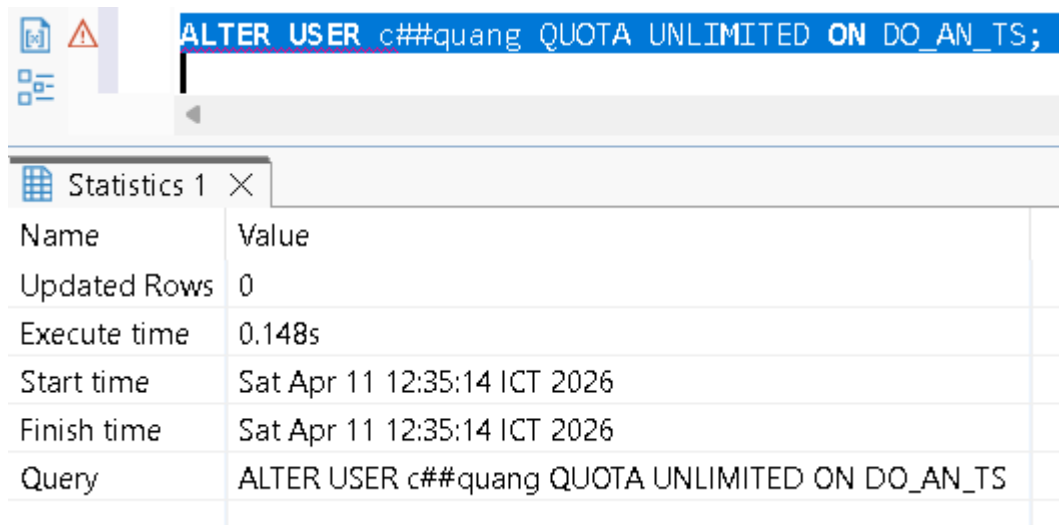
Thao tác này giúp hệ thống duy trì khả năng mở rộng linh hoạt và đảm bảo người dùng có đủ tài nguyên để thao tác dữ liệu.



Name	Value
Updated Rows	0
Execute time	1.031s
Start time	Sat Apr 11 12:34:49 ICT 2026
Finish time	Sat Apr 11 12:34:50 ICT 2026
Query	ALTER DATABASE DATAFILE '/u01/app/oracle/oradata/ORCL/do_an_ts_01.dbf' RESIZE 200M

Hình 5.3: Kết quả thực thi lệnh Alter Database Datafile.

(Nguồn: Nhóm sinh viên thực hiện)



ALTER USER c##quang QUOTA UNLIMITED ON DO_AN_TS;	
Statistics 1	
Name	Value
Updated Rows	0
Execute time	0.148s
Start time	Sat Apr 11 12:35:14 ICT 2026
Finish time	Sat Apr 11 12:35:14 ICT 2026
Query	ALTER USER c##quang QUOTA UNLIMITED ON DO_AN_TS

Hình 5.4: Kết quả thực thi lệnh cấp quyền Quota Unlimited.

(Nguồn: Nhóm sinh viên thực hiện)

5.2. Thực nghiệm kiểm soát TRANSACTION và tính toàn vẹn:

Một trong những nguyên lý cốt lõi của hệ quản trị cơ sở dữ liệu là đảm bảo tính toàn vẹn theo mô hình ACID. Oracle thực hiện điều này thông qua cơ chế kiểm soát giao dịch, cho phép xác nhận hoặc hủy bỏ thay đổi một cách linh hoạt.

5.2.1. Khởi tạo dữ liệu và chốt giao dịch (COMMIT):

5.2.1.1. Cú pháp lệnh:

```
INSERT INTO <tên_bảng> (...) VALUES (...);  
  
COMMIT;
```

5.2.1.2. Mô tả và thực thi:

Nhóm tiến hành chèn một bản ghi mô phỏng việc mở tài khoản ngân hàng cho khách hàng QUANG_SEC với số dư ban đầu là 100.000.000 VNĐ.

```
INSERT INTO tai_khoan_ngan_hang (stt, chu_tai_khoan, so_du)  
VALUES (1, 'QUANG_SEC', 100000000);
```

```
COMMIT;
```

Sau khi thực hiện COMMIT, dữ liệu được ghi nhận vĩnh viễn vào hệ thống thông qua cơ chế Redo Log. Điều này đảm bảo giao dịch được hoàn tất và không bị mất khi xảy ra sự cố.

The screenshot displays an Oracle SQL script editor with the following SQL commands:

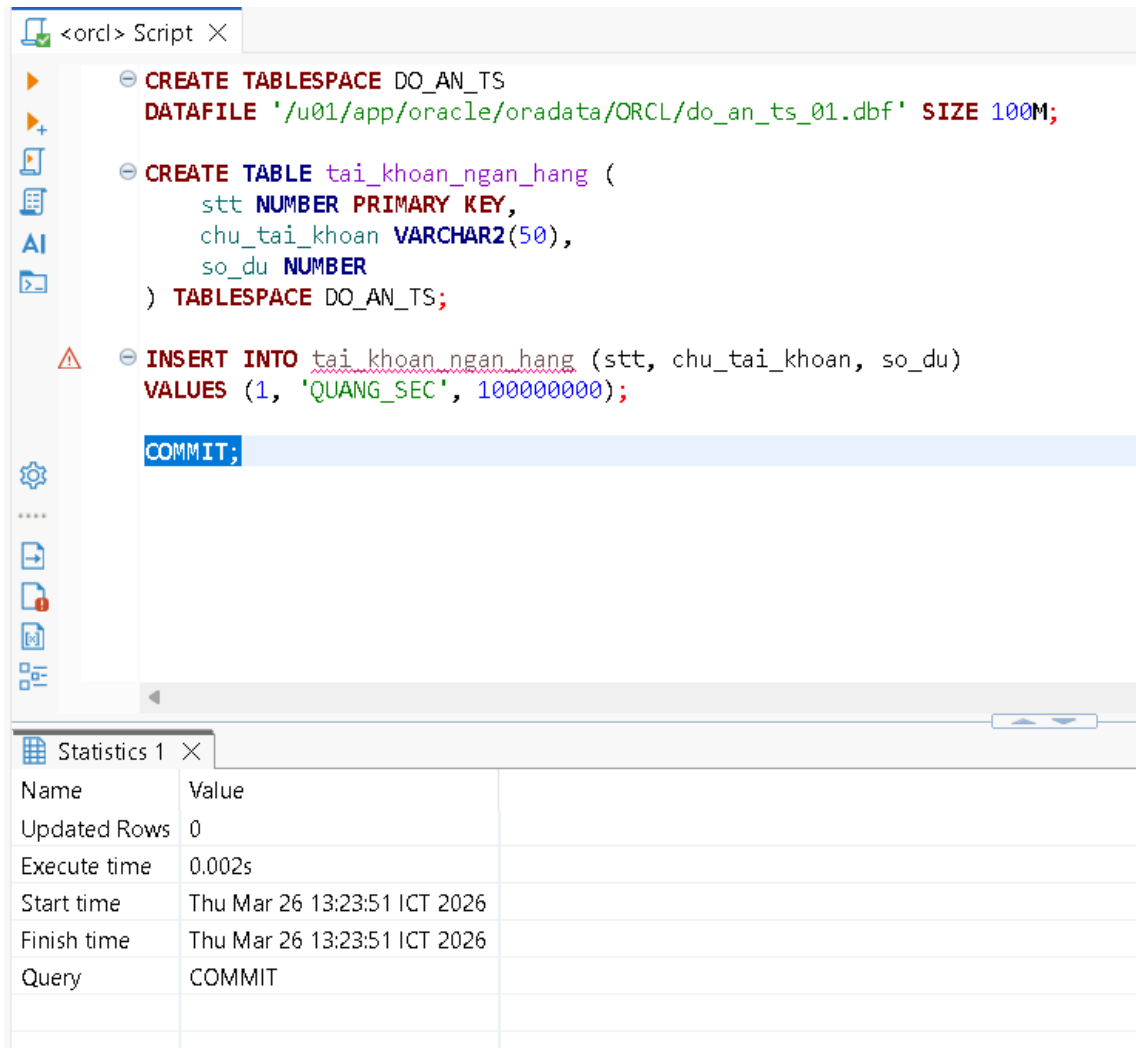
```
CREATE TABLESPACE DO_AN_TS  
DATAFILE '/u01/app/oracle/oradata/ORCL/do_an_ts_01.dbf' SIZE 100M;  
  
CREATE TABLE tai_khoan_ngan_hang (  
    stt NUMBER PRIMARY KEY,  
    chu_tai_khoan VARCHAR2(50),  
    so_du NUMBER  
) TABLESPACE DO_AN_TS;  
  
INSERT INTO tai_khoan_ngan_hang (stt, chu_tai_khoan, so_du)  
VALUES (1, 'QUANG_SEC', 100000000);  
  
COMMIT;
```

Below the script editor, a 'Statistics 1' window shows the execution details:

Name	Value
Updated Rows	1
Execute time	1.873s
Start time	Thu Mar 26 13:23:05 ICT 2026
Finish time	Thu Mar 26 13:23:07 ICT 2026
Query	INSERT INTO tai_khoan_ngan_hang (stt, chu_tai_khoan, so_du) VALUES (1, 'QUANG_SEC', 100000000)

Hình 5.5: Kết quả thực thi lệnh Insert.

(Nguồn: Nhóm sinh viên thực hiện)



Hình 5.6: Kết quả thực thi lệnh COMMIT.

(Nguồn: Nhóm sinh viên thực hiện)

5.2.2. Hủy bỏ giao dịch với SAVEPOINT và ROLLBACK:

5.2.2.1. Cú pháp lệnh:

```
SAVEPOINT <tên_điểm_lưu>;

ROLLBACK TO SAVEPOINT <tên_điểm_lưu>;
```

5.2.2.2. Mô tả và thực thi:

Để minh họa khả năng xử lý lỗi, nhóm tạo thêm tài khoản CHI_SEC và thiết lập một điểm lưu (SAVEPOINT). Sau đó thực hiện một thao tác UPDATE sai và tiến hành khôi phục lại dữ liệu.

```
INSERT INTO tai_khoan_ngan_hang (stt, chu_tai_khoan, so_du)
VALUES (2, 'CHI_SEC', 50000000);

COMMIT;

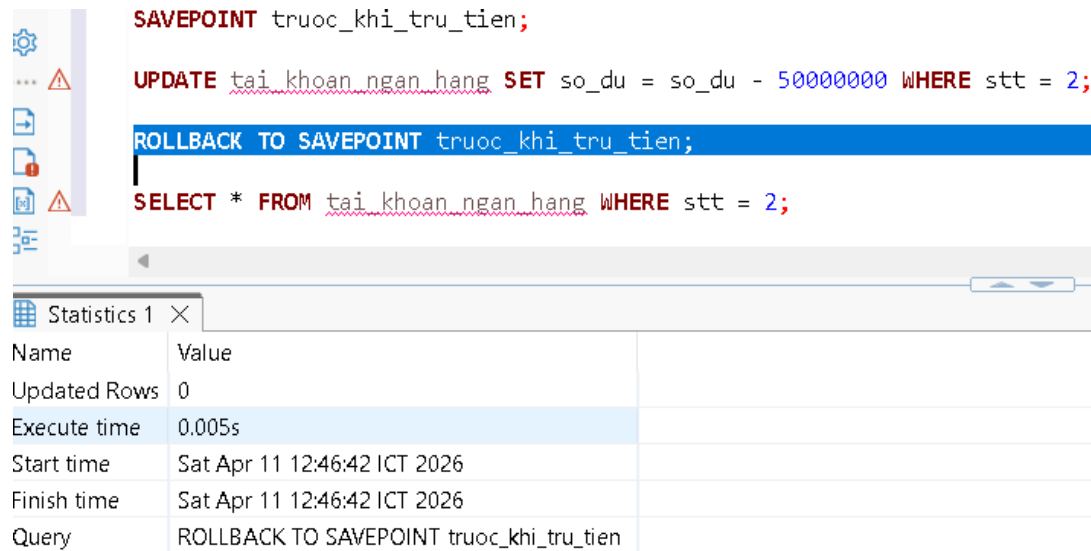
SAVEPOINT truoc_khi_tru_tien;

UPDATE tai_khoan_ngan_hang
SET so_du = so_du - 50000000
WHERE stt = 2;

ROLLBACK TO SAVEPOINT truoc_khi_tru_tien;

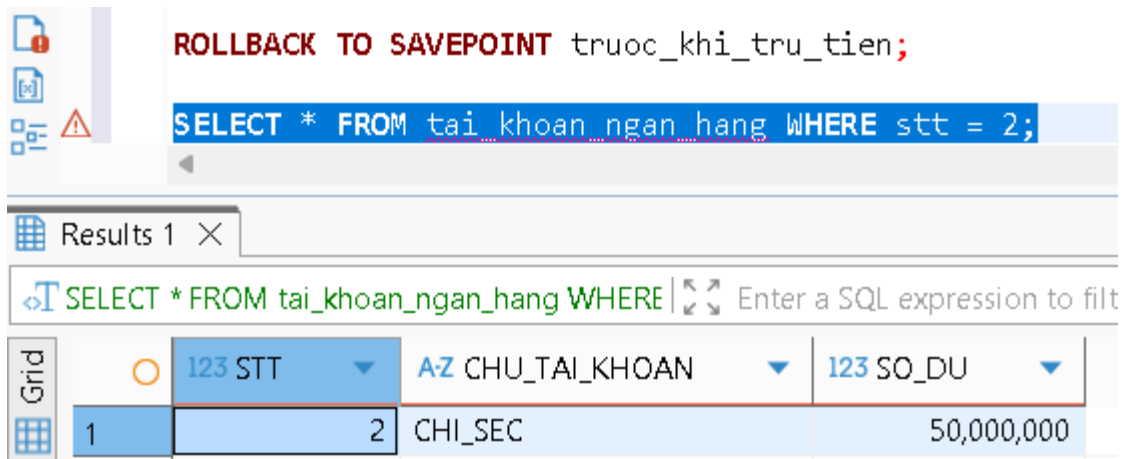
SELECT * FROM tai_khoan_ngan_hang WHERE stt = 2;
```

Kết quả cho thấy dữ liệu được khôi phục về trạng thái ban đầu trước khi xảy ra lỗi. Điều này chứng minh cơ chế Undo và Transaction Control của Oracle hoạt động hiệu quả trong việc bảo vệ tính toàn vẹn dữ liệu.



Hình 5.7: Quá trình thực thi lệnh ROLLBACK TO SAVEPOINT.

(Nguồn: Nhóm sinh viên thực hiện)



Hình 5.8: Kết quả khôi phục số dư tài khoản.

(Nguồn: Nhóm sinh viên thực hiện)

5.3. Thực nghiệm xử lý đồng thời (Concurrency) và hiện tượng chờ khóa (Blocking):

Trong các hệ thống cơ sở dữ liệu thực tế, nhiều người dùng có thể đồng thời truy cập và thao tác trên cùng một dữ liệu. Điều này đặt ra yêu cầu về cơ chế kiểm soát đồng thời nhằm đảm bảo tính nhất quán và tránh xung đột dữ liệu.

5.3.1. Cú pháp lệnh sử dụng:

```
UPDATE <tên_bảng>

SET <tên_cột> = <giá_trị_mới>

WHERE <điều_kiện>;
```

5.3.2. Kịch bản mô phỏng:

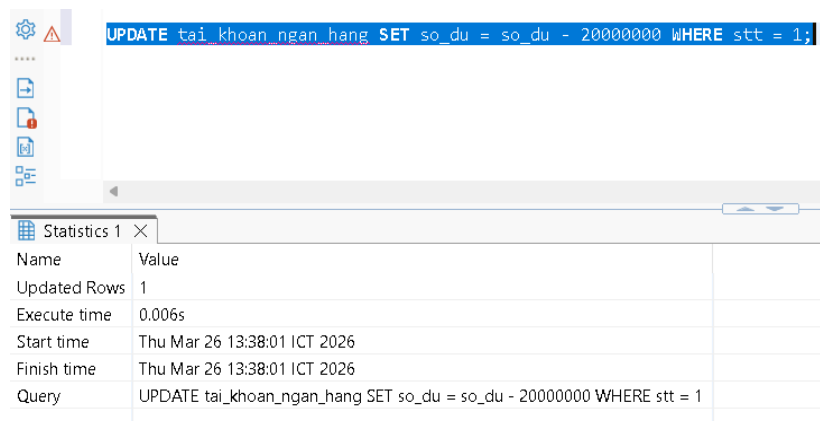
Nhóm tiến hành mở đồng thời hai phiên làm việc (Session A và Session B) và thực hiện cập nhật trên cùng một dòng dữ liệu (stt = 1).

5.3.2.1. Tại phiên A:

```
UPDATE tai_khoan_ngan_hang

SET so_du = so_du - 20000000

WHERE stt = 1
```



Hình 5.9: Lệnh Update tại Phiên A thiết lập khóa độc quyền (Row-level Lock).

(Nguồn: Nhóm sinh viên thực hiện)

5.3.2.2. Tại phiên B:

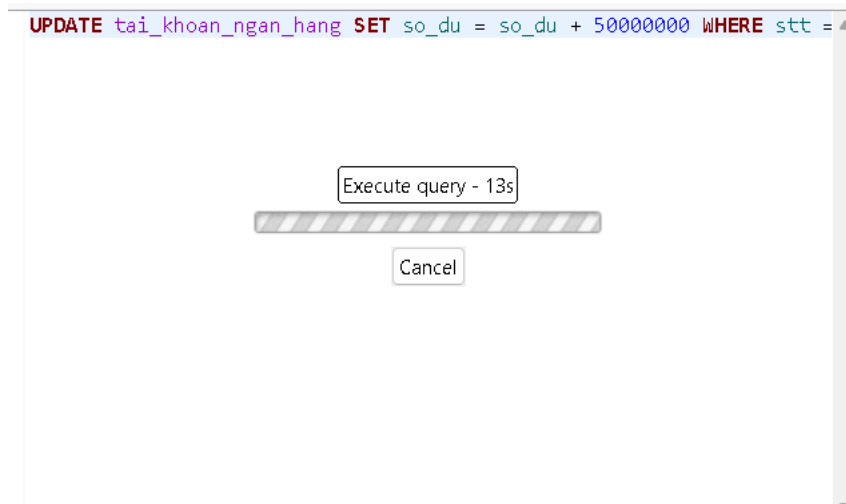
```
UPDATE tai_khoan_ngan_hang

SET so_du = so_du + 50000000
```

```
WHERE stt = 1;
```

5.3.3. Kết quả và đánh giá:

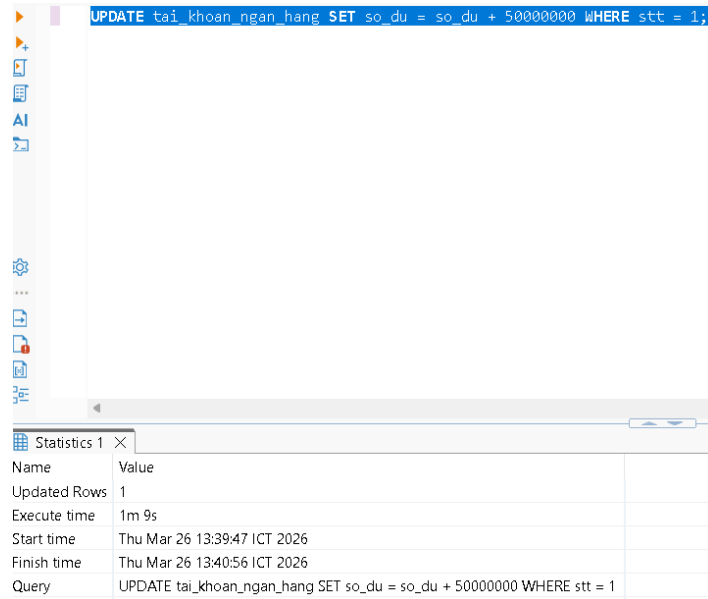
Khi phiên A thực hiện Update (chưa COMMIT), Oracle tự động áp dụng khóa độc quyền (Exclusive Lock) lên dòng dữ liệu. Do đó, câu lệnh Update tại phiên B không thể thực thi ngay mà bị đưa vào trạng thái chờ (Blocking).



Hình 5.10: Truy vấn bị chờ (Blocking) tại Phiên B.

(Nguồn: Nhóm sinh viên thực hiện)

Sau khi Phiên A thực hiện COMMIT, khóa được giải phóng và câu lệnh tại Phiên B tiếp tục được thực thi thành công.



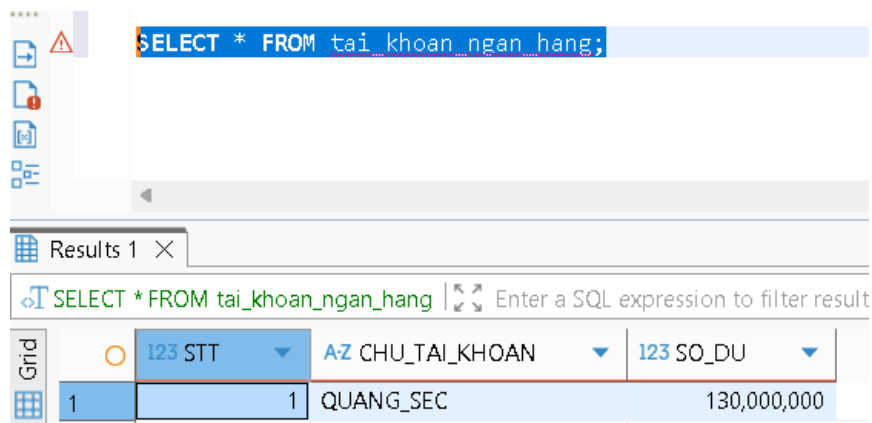
Hình 5.11: Lệnh tại Phiên B thực thi thành công ngay sau khi khóa được giải phóng.

(Nguồn: Nhóm sinh viên thực hiện)

Kiểm tra kết quả:

```
SELECT * FROM tai_khoan_ngan_hang;
```

Kết quả trả về số dư chính xác là 130.000.000 VNĐ (100 triệu - 20 triệu + 50 triệu). Điều này chứng minh các giao dịch được thực thi tuần tự, không xảy ra hiện tượng ghi đè dữ liệu. Qua đó đảm bảo tính nhất quán dữ liệu trong môi trường nhiều người dùng, đồng thời hạn chế các hiện tượng như Lost Update.



Hình 5.12: Kết quả sau khi giải phóng khóa và thực thi thành công.

(Nguồn: Nhóm sinh viên thực hiện)

5.4. Thực nghiệm hiện tượng Deadlock (Khóa chết):

Deadlock là trạng thái xảy ra khi hai hoặc nhiều giao dịch giữ tài nguyên và chờ lẫn nhau, tạo thành vòng lặp bế tắc. Đây là một trong những tình huống nghiêm trọng trong hệ thống cơ sở dữ liệu.

5.4.1 Kịch bản mô phỏng:

Nhóm thực hiện các thao tác đan chéo giữa hai phiên làm việc:

- Bước 1 – Phiên A:

```
UPDATE tai_khoan_ngan_hang  
SET so_du = so_du + 1  
WHERE stt = 1;
```

- Bước 2 – Phiên B:

```
UPDATE tai_khoan_ngan_hang  
SET so_du = so_du + 1  
WHERE stt = 2;
```

- Bước 3 – Phiên A:

```
UPDATE tai_khoan_ngan_hang  
SET so_du = so_du + 1  
WHERE stt = 2;
```

- Bước 4 – Phiên B:

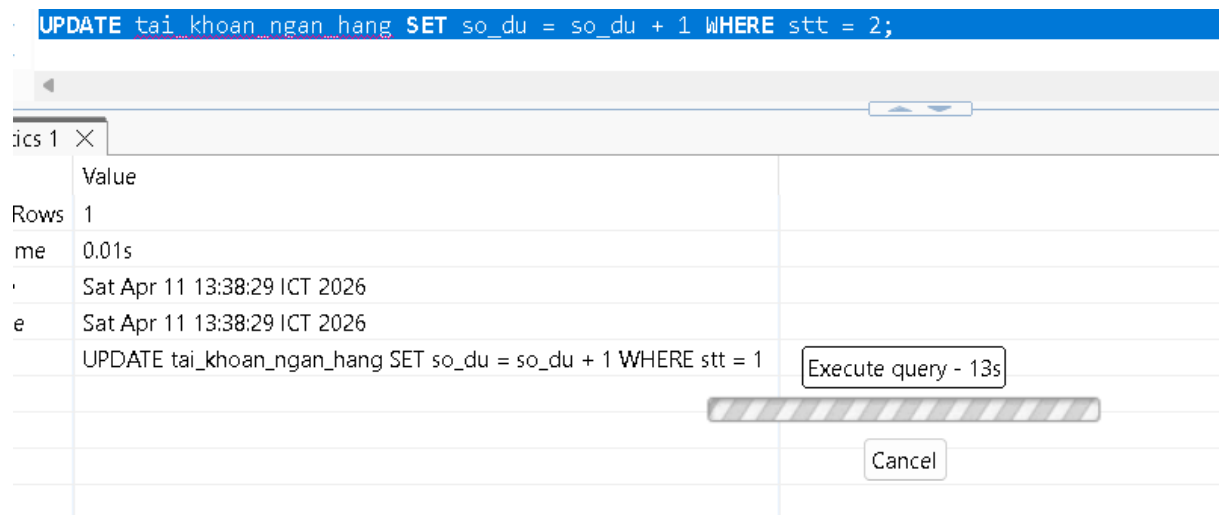
```
UPDATE tai_khoan_ngan_hang  
SET so_du = so_du + 1  
WHERE stt = 1;
```

5.4.2. Kết quả và đánh giá:

Tại bước 4, Oracle phát hiện vòng lặp chờ giữa hai phiên và xác định đây là tình huống Deadlock. Hệ thống tự động can thiệp bằng cách hủy (ROLLBACK) một trong hai giao dịch và trả về lỗi:

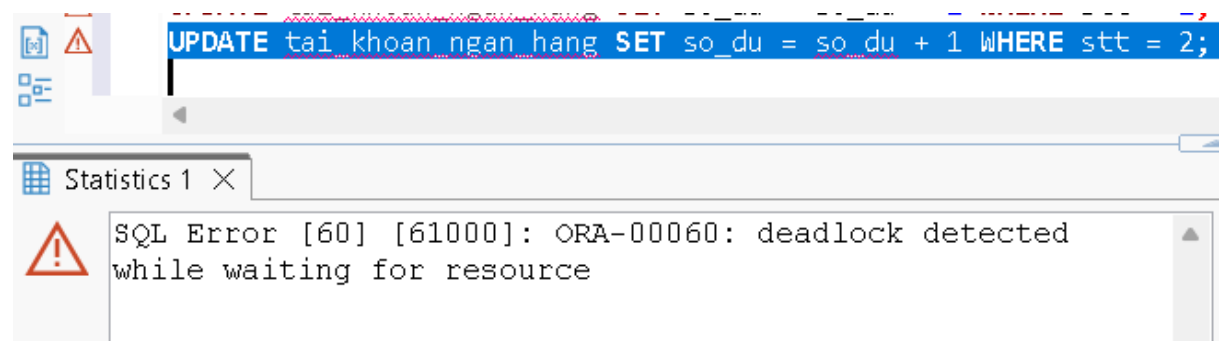
```
ORA-00060: deadlock detected while waiting for resource
```

Nhờ cơ chế phát hiện và xử lý Deadlock tự động, hệ thống tránh được trạng thái treo vô hạn và tiếp tục hoạt động bình thường.



Hình 5.13: Truy vấn bị treo do cập nhật chéo.

(Nguồn: Nhóm sinh viên thực hiện)



Hình 5.14: Oracle phát hiện deadlock và hủy giao dịch.

(Nguồn: Nhóm sinh viên thực hiện)

5.5. Nhận xét và đánh giá tổng quan:

Thông qua toàn bộ quá trình thực nghiệm, hệ quản trị cơ sở dữ liệu Oracle 19c đã chứng minh được khả năng vận hành ổn định và thông minh trong môi trường doanh nghiệp có nhiều giao dịch đồng thời.

Về quản trị tài nguyên, cơ chế Tablespace cho phép tách biệt rõ ràng giữa logic và vật lý, giúp hệ thống dễ dàng mở rộng và kiểm soát dung lượng lưu trữ (Zero Downtime). Về kiểm soát giao dịch, sự kết hợp giữa COMMIT, ROLLBACK và SAVEPOINT đảm bảo dữ liệu luôn tuân thủ nguyên lý ACID, cho phép khôi phục an toàn khi xảy ra sự cố thao tác.

Đặc biệt, ở khía cạnh xử lý đồng thời (Concurrency Control), đề minh chứng rõ nét cho cơ sở lý thuyết ở Chương 4, nhóm đã tổng hợp lại các thông số thời gian thực thi (Execute time) đo lường được trên DBeaver:

Đánh giá cơ chế nội bộ của Oracle	Thời gian thực thi	Đánh giá cơ chế nội bộ của Oracle
Phiên A: Cập nhật dòng stt = 1	0.006s	Row-level Lock: Lệnh chạy thành công ngay lập tức, Oracle cấp khóa độc quyền cho Phiên A
Phiên B: Cập nhật trùng dòng stt = 1	Bị treo (13s)	Blocking: Oracle ngăn chặn lỗi ghi đè, tự động ép truy vấn B vào trạng thái xếp hàng chờ

Phiên B: Sau khi A nhả khóa (COMMIT)	1m 9s (bao gồm thời gian chờ khóa + thực thi thực tế)	Consistency: Phần lớn là thời gian chờ khóa (Blocking); sau khi Phiên A COMMIT, lệnh thực thi ngay và số dư cộng dồn chính xác
Phiên A & B: Cập nhật chéo tài nguyên	Báo lỗi lập tức	Deadlock Detection: Tiến trình nền chủ động phát hiện bế tắc, hủy 1 giao dịch và văng lỗi ORA-00060

Bảng 5.1: Đối chiếu kết quả thực nghiệm xử lý xung đột.

Từ đó kết luận được rằng, các kết quả bằng số liệu thực tế này hoàn toàn khớp với cơ sở lý thuyết. Qua đó, khẳng định Oracle không chỉ đảm bảo tính toàn vẹn dữ liệu tuyệt đối mà còn cung cấp một bộ máy giải quyết xung đột vô cùng chặt chẽ, đáp ứng tốt cho các hệ thống có lượng truy xuất đồng thời khổng lồ.

C. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Qua quá trình phân tích lý thuyết kết hợp với thực nghiệm trên môi trường Oracle 19c, có thể nhận thấy rằng việc nắm vững các cơ chế quản trị không gian lưu trữ và kiểm soát đồng thời đóng vai trò nền tảng trong việc đảm bảo hiệu năng và độ tin cậy của hệ quản trị cơ sở dữ liệu. Tablespace không chỉ là một đơn vị tổ chức lưu trữ logic mà còn là công cụ giúp tối ưu hóa phân bổ tài nguyên, hỗ trợ mở rộng hệ thống và nâng cao hiệu quả quản trị. Song song đó, cơ chế Lock cùng với các thành phần liên quan như Transaction, Undo và MVCC đã chứng minh khả năng điều phối truy cập dữ liệu một cách chặt chẽ, hạn chế xung đột và duy trì tính nhất quán trong môi trường có nhiều giao dịch đồng thời.

Các kết quả thực nghiệm đã làm rõ cách Oracle xử lý hiệu quả các tình huống thực tế như blocking và deadlock, qua đó khẳng định rằng hệ thống không chỉ đảm bảo tính toàn vẹn dữ liệu theo nguyên lý ACID mà còn duy trì được sự ổn định khi chịu tải cao. Điều này cho thấy, việc hiểu và vận dụng đúng các cơ chế nội tại không chỉ mang ý nghĩa lý thuyết mà còn có giá trị ứng dụng trực tiếp trong thiết kế và vận hành hệ thống.

Tuy nhiên, hiệu năng tổng thể của một hệ thống cơ sở dữ liệu không chỉ phụ thuộc vào cơ chế bên trong của hệ quản trị, mà còn chịu ảnh hưởng đáng kể từ cách tổ chức và triển khai ở tầng ứng dụng. Do đó, một hướng phát triển quan trọng là tối ưu hóa logic xử lý trong ứng dụng nhằm giảm thiểu xung đột và tránh gây nghẽn hệ thống. Cụ thể, cần kiểm soát phạm vi và thời gian tồn tại của transaction, hạn chế giữ khóa trong thời gian dài, thiết kế truy vấn theo hướng truy cập có chọn lọc, đồng thời đảm bảo thứ tự thao tác nhất quán để giảm nguy cơ phát sinh deadlock. Bên cạnh đó, việc kết hợp các chiến lược như phân tách nghiệp vụ, xử lý bất đồng bộ hoặc sử dụng hàng đợi (queue) trong các tác vụ có tần suất cao cũng góp phần nâng cao khả năng mở rộng và ổn định của hệ thống.

Từ những phân tích trên, có thể khẳng định rằng sự phối hợp hiệu quả giữa quản trị cơ sở dữ liệu và thiết kế ứng dụng là yếu tố quyết định để xây dựng các hệ thống thông tin có khả năng vận hành bền vững trong môi trường thực tế với cường độ truy cập lớn. Đây cũng là nền tảng để tiếp tục nghiên cứu các giải pháp tối ưu sâu hơn, hướng

đến các hệ thống có hiệu năng cao, khả năng chịu lỗi tốt và thích ứng linh hoạt với sự phát triển của dữ liệu trong tương lai.

TÀI LIỆU THAM KHẢO

- Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th ed.). Pearson.
- Oracle. (2019). *Oracle Database Performance Tuning Guide, 19c (E96203 - 02)*. Oracle.
- Oracle Corporation. (2019). *Oracle Database 19c Concepts*. Oracle Documentation Library.
- Oracle Corporation. (2023). *Oracle Data Guard Concepts and Administration*.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/sbydb/>
- Oracle Corporation. (2023). *Oracle Database 19c Documentation*.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/>
- Oracle Corporation. (2023). *Oracle Database Administrator's Guide 19c*.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/admin/>
- Oracle Corporation. (2023). *Oracle Database Security Guide 19c*.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/dbseg/>
- Oracle Corporation. (2025). *Oracle Database Concepts 19c (19.29 release update)*. Oracle.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/cncpt/index.html>
- Oracle Corporation. (2025). *Oracle Database Performance Tuning Guide 19c (19.29 release update)*. Oracle.
<https://docs.oracle.com/en/database/oracle/oracle-database/19/tgdba/index.html>
- Red Hat Inc. (2023). *Red Hat Enterprise Linux 9 Documentation*.
https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/9
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2020). *Database System Concepts* (7th ed.). McGraw - Hill Education.

PHỤ LỤC

BẢNG PHÂN CÔNG CÔNG VIỆC NHÓM

Họ và tên	Công việc đảm nhiệm	Tỷ lệ đóng góp
Nguyễn Minh Quang (<i>Trưởng nhóm</i>)	- Trình bày Chương 4: Cơ chế khóa (Lock) và điều khiển đồng thời. - Phân tích các loại Lock, hiện tượng blocking và deadlock. - Tham gia thực nghiệm: Xây dựng kịch bản concurrency, mô phỏng blocking và deadlock trên nhiều session và ghi nhận kết quả. - Thu thập hình ảnh minh họa quá trình cài đặt và kiểm thử hệ thống để phục vụ cho việc trình bày nội dung thực hành trong tiểu luận.	100%
Nguyễn Ngọc Quỳnh Chi	- Thực hiện phần mở đầu và kết luận của tiểu luận, xác định mục tiêu nghiên cứu và tổng hợp giá trị thực tiễn của đề tài. - Rà soát, chỉnh sửa và hoàn thiện toàn bộ nội dung nhằm đảm bảo tính thống nhất về bố cục và văn phong học thuật.	100%

	- Tham gia thực nghiệm trên Oracle 19c: kiểm tra hoạt động hệ thống, thực hiện các truy vấn SQL cơ bản và hỗ trợ tổng hợp kết quả thực nghiệm của nhóm.	
Nguyễn Ngọc Tuyết Ngân	- Trình bày Chương 3: Quản lý giao dịch (Transaction) và tính toàn vẹn dữ liệu. - Phân tích các cơ chế Undo, Redo và các lệnh của COMMIT, ROLLBACK, SAVEPOINT. - Tham gia thực nghiệm: xây dựng và chạy các kịch bản kiểm soát transaction, kiểm tra khả năng phục hồi dữ liệu và ghi nhận kết quả minh họa.	100%
Nguyễn Đức Thuận	- Trình bày Chương 5: Triển khai thực nghiệm và đánh giá. - Tổng hợp các kịch bản thực nghiệm liên quan đến Tablespace, Transaction và Lock. - Tham gia trực tiếp vào toàn bộ quá trình thực thi trên Oracle 19c, thu thập kết quả, hình ảnh minh họa và phân tích đối chiếu với lý thuyết.	100%
Nguyễn Thị Phương Trâm	- Trình bày Chương 2: Quản trị Tablespace trong Oracle.	100%

	- Phân tích vai trò, phân loại Tablespace và cơ chế quản lý dung lượng, quota. - Tham gia thực nghiệm: thực hiện tạo Tablespace, mở rộng Datafile và kiểm tra phân bổ dung lượng trên hệ thống.	
Huỳnh Hồng Ánh Tuyết	- Trình bày Chương 1: Tổng quan kiến trúc logic và cơ chế đồng thời. - Phân tích nguyên lý ACID và cơ chế MVCC trong Oracle. - Tham gia thực nghiệm: hỗ trợ kiểm tra các cơ chế đọc nhất quán, quan sát hành vi hệ thống khi có nhiều truy vấn đồng thời.	100%